Docker Challenge

1.Create a customized Docker image using a Dockerfile.

Task: Create a customized Docker image using a Dockerfile, build the image, run the container, and verify the application.

Date: October 15, 2026

Application: Python Flask Web Application

Environment: AWS EC2 Ubuntu + Docker

1. Objective

The objective of this task is to create a custom Docker image for a Python Flask application using a Dockerfile and deploy it as a container.

We used the Python application provided by the trainer:

GitHub — betawins/Python-app

2. Environment details

Component	Details
Cloud provider	AWS
Server	EC2 Ubuntu
Docker version	29.9.0
Application	Python Flask

Base image	`python:3.12-slim`
Docker image	`python-challenge:v1`
Docker container	`python-challenge-container`
Host port	`8090`
Application port	`8080`
Application color	Green

3. Architecture and workflow

GitHub Repository

Python Flask source code

Custom Dockerfile

Python 3.12 + Flask + app files

Docker Image

python-challenge:v1

Docker Container

8090 → 8080

Browser / curl

HTTP 200 OK

4. Step-by-step practical implementation

Step 1 — Connect to EC2 and verify Docker

Connected to the Ubuntu EC2 instance and switched to root:

sudo su

Verified Docker:

docker --version

Actual output:

Docker version 29.9.0, build f415da8

Checked running containers:

docker ps

The server already had Jenkins, WordPress, and MySQL running.

Checked available disk space:

df -h /

Actual output:

Filesystem	Size	Used	Avail	Use%
/dev/root	28G	13G	15G	47%

Purpose: Confirm Docker is running and enough space is available before building a new image.

Step 2 — Clone the Python application

Created the project directory:

mkdir -p /home/ubuntu/docker-challenge/task1
cd /home/ubuntu/docker-challenge/task1

Cloned the trainer's repository:

git clone https://github.com/betawins/Python-app.git

Entered the repository and checked its files:

cd Python-app

ls -la

The project contained:

Python-app/
├── app.py
├── requirements.txt
├── Dockerfile
├── Jenkinsfile
├── templates/
└── .gitignore

Step 3 — Understand the source code

We inspected the application:

cat app.py

cat requirements.txt

cat Dockerfile

Our observations:

File	Purpose
`app.py`	Runs the Flask web application
`requirements.txt`	Contains the Flask dependency
`templates/hello.html`	Renders the webpage

`Dockerfile`	Original build instructions
`Jenkinsfile`	CI/CD pipeline definition

The original Dockerfile used `python:3.6`. We created a customized Dockerfile using Python 3.12 Slim instead.

The Flask application listens on port `8080`, and its background color can be controlled using the `APP_COLOR` environment variable.

Step 4 — Create a customized Dockerfile

Created `Dockerfile.custom`:

```
cat > Dockerfile.custom <<'EOF'
FROM python:3.12-slim

LABEL maintainer="Chandrashekar"

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY templates/ ./templates/

ENV APP_COLOR=green

EXPOSE 8080

CMD ["python", "app.py"]
EOF
```

Verified:

cat Dockerfile.custom

Dockerfile explanation

Instruction	Purpose
`FROM python:3.12-slim`	Uses a Python 3.12 slim base image
`LABEL`	Adds image metadata
`WORKDIR /app`	Sets the working directory
`COPY requirements.txt .`	Copies the dependency list
`RUN pip install`	Installs Flask and required packages
`COPY app.py .`	Copies Python application code
`COPY templates/`	Copies HTML templates
`ENV APP_COLOR=green`	Sets the default background color
`EXPOSE 8080`	Documents the application's listening port

`CMD ["python", "app.py"]`	Starts the Flask application

Step 5 — Build the Docker image

Executed:

docker build -f Dockerfile.custom -t python-challenge:v1 .

Verified:

docker image ls python-challenge

Actual result:

IMAGE ID DISK USAGE CONTENT SIZE
python-challenge:v1 7ef169aee875 198MB 48.3MB

Result: The Docker image was created successfully.

Step 6 — Run the Docker container

Executed:

```
docker run -d \
  --name python-challenge-container \
  -p 8090:8080 \
  -e APP_COLOR=green \
  python-challenge:v1
```

Command explanation:

Option	Purpose
`-d`	Runs the container in the background

`--name`	Assigns a container name
`-p 8090:8080`	Maps EC2 port 8090 to container port 8080
`-e APP_COLOR=green`	Sets the application background color
`python-challenge:v1`	Specifies the Docker image

We selected host port `8090` because the existing Jenkins container was already using host port `8080`.

Step 7 — Verify container status

Executed:

docker ps --filter "name=python-challenge-container"

Actual result:

CONTAINER ID IMAGE STATUS PORTS

45941f5b03a8 python-challenge:v1 Up 0.0.0.0:8090->8080/tcp

Result: Container running successfully.

Step 8 — Test the Flask application

Executed:

curl -i http://localhost:8090/

Actual response:

HTTP/1.1 200 OK

Server: Werkzeug/3.1.9 Python/3.12.15

Content-Type: text/html; charset=utf-8

The HTML response contained:

<title>Hello from Flask</title>

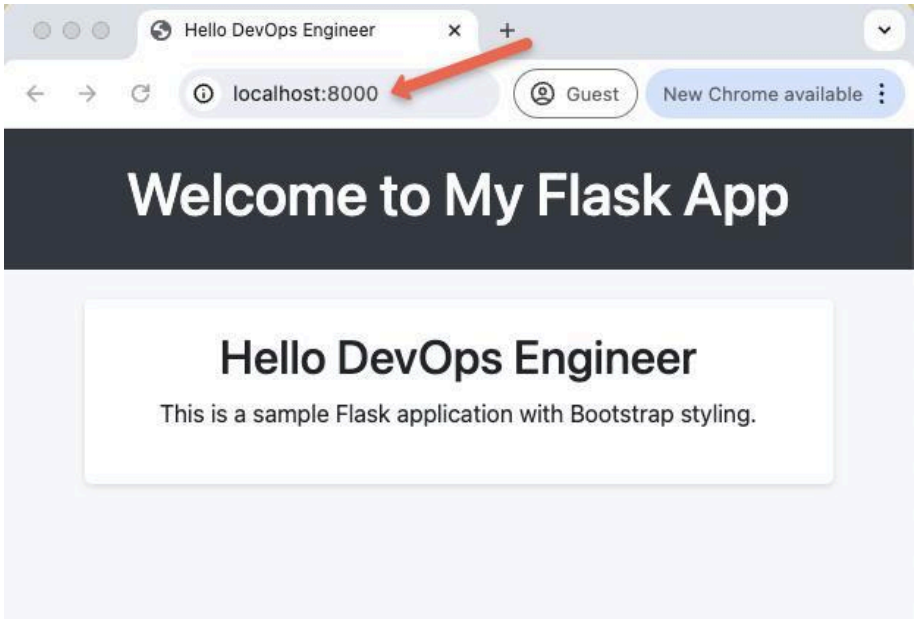
<body style="background: #16a085;"></body>

<h1>Hello, s3cloudhub</h1>

Result: The Python application responded successfully.

Step 9 — Verify the application in a browser

We opened the running application in the browser using the EC2 public IP and port `8090`.



devopscube.com

Illustrative view of the Flask application's green-background webpage. Your browser screenshot independently confirmed the same successful result.

my screenshot confirmed that the webpage loaded and displayed:

Hello, s3cloudhub

The browser showed the expected green background.

5. Final verification results

Python Flask Docker Application

SUCCESS

Image tag

v1

HTTP status

200 OK

Source code cloned successfully

Custom Dockerfile created

Docker image built

Container started

curl test successful

Browser verification successful

6. Real-time interview questions

Q1. How do you create a customized Docker image?

I create a Dockerfile containing instructions to select the base image, install dependencies, copy application files, expose the required port, and configure the startup command. Then I build it using `docker build`.

Q2. What is the difference between Docker image and container?

A Docker image is a packaged application template. A container is a running instance created from that image.

Q3. Why did you use `python:3.12-slim`?

It provides a more recent supported Python runtime with fewer unnecessary packages than a full Python image.

Q4. What is the difference between `EXPOSE` and `-p`?

`EXPOSE` documents the port an application uses inside the image. `-p` publishes a container port through a host port when starting a container.

Q5. Why use `--no-cache-dir` with pip?

It prevents pip from storing downloaded package caches in the image, helping reduce unnecessary image content.

Q6. What happens if the container stops?

I inspect the container status and logs using `docker ps -a` and `docker logs`, identify the error, and correct the application or configuration.

Q7. How can you change the background color without rebuilding the image?

By passing a different `APP_COLOR` environment variable when starting the container:

```
docker run -d -p 8091:8080 \
  -e APP_COLOR=blue \
  python-challenge:v1
```

This example uses a different host port to avoid conflicting with the existing container.

Q8. Is Flask's built-in development server suitable for production?

No. For production deployment, a suitable WSGI application server such as Gunicorn should be used, along with secure container and network configuration.

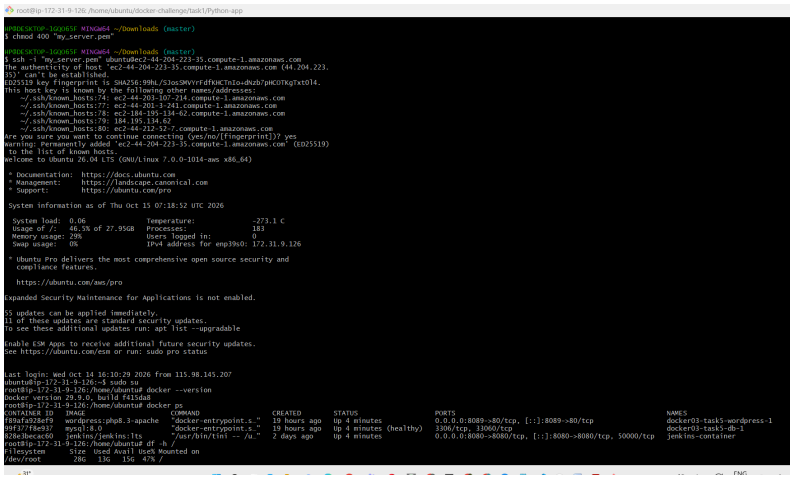
7. How to explain this project in an interview

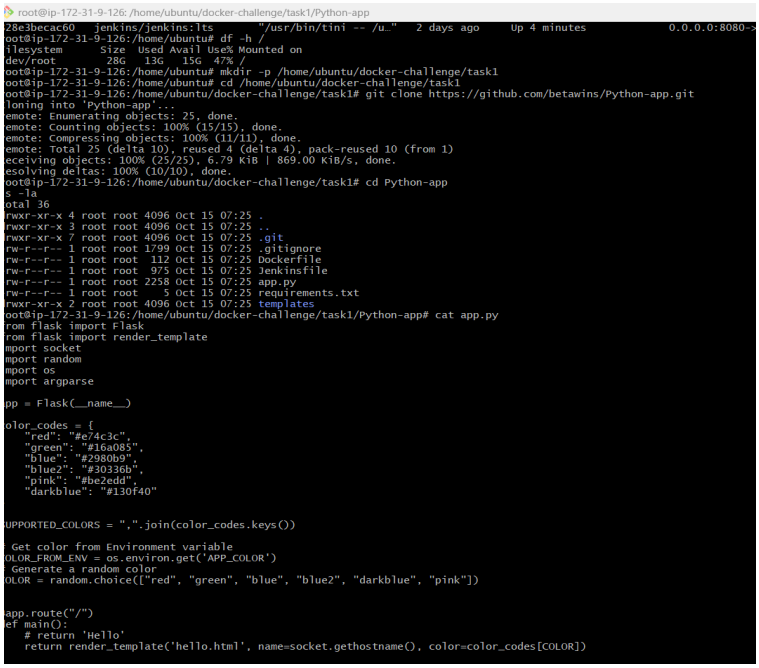
> I worked on a Docker containerization challenge using a Python Flask application.
>
> I cloned the application source code from GitHub and analyzed its dependencies, application port, and startup command.
>
> Then I created a customized Dockerfile using Python 3.12 Slim as the base image, installed the Python dependencies, copied the application source and HTML templates, and configured the application to run on port 8080.
>
> I built the Docker image with the tag `python-challenge:v1` and started a container using port mapping `8090:8080`.
>
> Finally, I verified the container was running, tested the application using curl, received HTTP 200 OK, and successfully accessed the application in my browser.

8. Conclusion

Docker Challenge — Task 1 successfully completed.

You demonstrated that you can inspect application source code, create a Dockerfile, build a custom image, run a container, configure environment variables and port mapping, and verify an application using curl and a browser.





root@ip-172-31-9-126: /home/ubuntu/docker-challenge/task1/Python-app

```
def main():
    # return 'Hello'
    return render_template('hello.html', name=socket.gethostname(), color=color_codes[COLOR])


if __name__ == "__main__":

    print(" This is a sample web application that displays a colored background. \n"
          " A color can be specified in two ways. \n"
          "\n"
          " 1. As a command line argument with --color as the argument. Accepts one of " + SUPPORTED_COLORS + " \n"
          " 2. As an Environment variable APP_COLOR. Accepts one of " + SUPPORTED_COLORS + " \n"
          " 3. If none of the above then a random color is picked from the above list. \n"
          " Note: Command line argument precedes over environment variable.\n"
          "\n"
          "")

    # Check for Command Line Parameters for color
    parser = argparse.ArgumentParser()
    parser.add_argument('--color', required=False)
    args = parser.parse_args()

    if args.color:
        print("Color from command line argument =" + args.color)
        COLOR = args.color
        if COLOR_FROM_ENV:
            print("A color was set through environment variable -" + COLOR_FROM_ENV + ". However, color from command line argument takes precendence.")
    elif COLOR_FROM_ENV:
        print("No Command line argument. Color from environment variable =" + COLOR_FROM_ENV)
        COLOR = COLOR_FROM_ENV
    else:
        print("No command line argument or environment variable. Picking a Random Color =" + COLOR)

    # Check if input color is a supported one
    if COLOR not in color_codes:
        print("Color not supported. Received '" + COLOR + "' expected one of " + SUPPORTED_COLORS)
        exit(1)

    # Run Flask Application
    app.run(host="0.0.0.0", port=8080)root@ip-172-31-9-126:/home/ubuntu/docker-challenge/task1/Python-app# cat requirements.txt
Flaskroot@ip-172-31-9-126:/home/ubuntu/docker-challenge/task1/Python-app# cat Dockerfile
FROM python:3.6

RUN pip install flask

COPY . /opt/

EXPOSE 8080

WORKDIR /opt

ENTRYPOINT ["python", "app.py"]root@ip-172-31-9-126:/home/ubuntu/docker-challenge/task1/Python-app# |
cat > Dockerfile.custom <<'EOF'
FROM python:3.12-slim

LABEL maintainer="Chandrashekar"

WORKDIR /app
```

root@ip-172-31-9-126:/home/ubuntu/docker-challenge/task1/Python-app

```
COPY templates/ ./templates/

ENV APP_COLOR=green

EXPOSE 8080

CMD ["python", "app.py"]
root@ip-172-31-9-126:/home/ubuntu/docker-challenge/task1/Python-app# docker build -f Dockerfile.custom -t python-challenge:v1 .
[+] Building 5.3s (12/12) FINISHED                                                
```

```
root@ip-172-31-9-126:/home/ubuntu/docker-challenge/task1/Python-app# docker image ls python-challenge

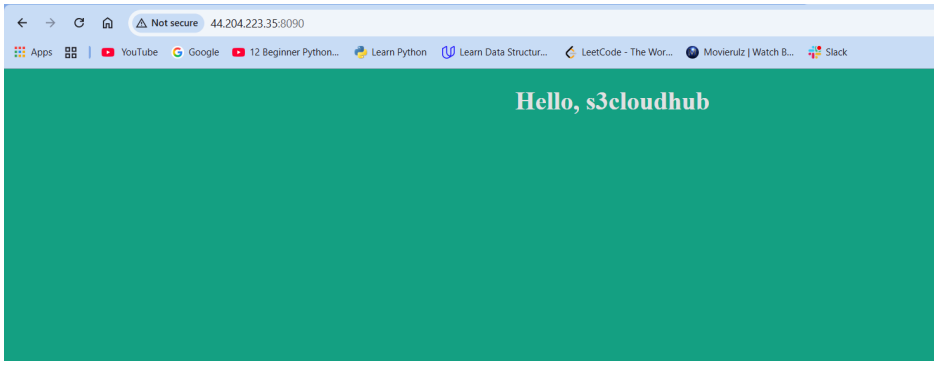
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
python-challenge:v1   7ef169aee875       198MB         48.3MB
root@ip-172-31-9-126:/home/ubuntu/docker-challenge/task1/Python-app# docker run -d \
  --name python-challenge-container \
  -p 8090:8080 \
  -e APP_COLOR=green \
  python-challenge:v1
45941f5b03a837e89b2b0769a130740a916ba1642af58847bab6626aca77ec8a
root@ip-172-31-9-126:/home/ubuntu/docker-challenge/task1/Python-app# docker ps --filter "name=python-challenge-container"
CONTAINER ID   IMAGE                COMMAND           CREATED         STATUS         PORTS                                                  NAMES
45941f5b03a8   python-challenge:v1   "python app.py"   13 seconds ago   Up 12 seconds   0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp   python-challenge-container
root@ip-172-31-9-126:/home/ubuntu/docker-challenge/task1/Python-app# curl -i http://localhost:8090/
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.12.15
Date: Thu, 15 Oct 2026 07:35:19 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 235
Connection: close

<!doctype html>
<title>Hello from Flask</title>
<body style="background: #16a085;"></body>
<div style="color: #e4e4e4;
    text-align:  center;
    height: 90px;
    vertical-align:  middle;">

  <h1>Hello, s3cloudhub</h1>



</div>root@ip-172-31-9-126:/home/ubuntu/docker-challenge/task1/Python-app#
```



1. Push the image to Docker Hub.

objective

The objective of Task 2 is to push the customized Docker image created in Task 1 to Docker Hub so that the same image can be downloaded and used on other Docker servers.

In Task 1, we created and tested:

`python-challenge:v1`

In Task 2, we published it as:

`chandrashekar8639211489/python-challenge:v1`

2. Environment details

Component	Actual configuration
Cloud	AWS EC2
Operating system	Ubuntu
Docker version	29.9.0
Application	Python Flask
Local Docker image	`python-challenge:v1`
Image ID	`7ef169aee875`
Docker Hub username	`chandrashekar863921148 9`
Repository	`python-challenge`
Image tag	`v1`
Repository visibility	Public

Docker Hub authentication	Successful

3. Architecture and workflow

Ubuntu EC2 Server

python-challenge:v1

Docker Tag

Add Docker Hub username

Docker Login

Authenticate with Docker Hub

Docker Hub Registry

python-challenge:v1 published

4. Step-by-step practical implementation

Step 1 — Create a Docker Hub repository

Opened Docker Hub Repositories and created a repository using these values:

Field	Value
Namespace	`chandrashekar8639211489`
Repository name	`python-challenge`

Visibility	Public
Description	Python Flask Docker Image — DevOps Challenge

The repository creation was verified using a Docker Hub screenshot.

Purpose: A Docker Hub repository stores image versions so other systems and servers can pull those images.

Step 2 — Verify the existing Docker image

On the Ubuntu EC2 server, we already had the image created in Task 1:

```
docker image ls python-challenge
```

Actual output:

```
IMAGE                ID             DISK USAGE   CONTENT SIZE
python-challenge:v1  7ef169aee875   198MB        48.3MB
```

Purpose: Confirm that the image exists locally before attempting to upload it.

Step 3 — Tag the Docker image

Executed:

```
docker tag python-challenge:v1 \
  chandrashekar8639211489/python-challenge:v1
```

Verified the local image:

```
docker image ls python-challenge
docker image ls chandrashekar8639211489/python-challenge
```

Actual output:

```
IMAGE                                ID
python-challenge:v1                  7ef169aee875
```

chandrashekar8639211489/python-challenge:v1 7ef169aee875

Purpose: The `docker tag` command assigns the Docker Hub namespace and repository name to the existing image.

Both image references have the same Image ID because tagging does not rebuild the image.

Step 4 — Log in to Docker Hub

Executed:

docker login -u chandrashekar8639211489

Docker requested authentication.

Password:

Login Succeeded

Result: Successfully authenticated to Docker Hub.

Security: Use a Personal Access Token (PAT) rather than storing an account password in scripts. Never include access tokens in GitHub repositories or documentation.

Step 5 — Push the image to Docker Hub

Executed:

docker push chandrashekar8639211489/python-challenge:v1

Actual output:

The push refers to repository [docker.io/chandrashekar8639211489/python-challenge]

55de361ca85b: Pushed

ecc510c1e359: Mounted from library/wordpress

75b6a36c64a1: Pushed

44136fa355b3: Mounted from chandrashekar8639211489/chandu-nginx

6912e23eb445: Pushed

bbc5a1223747: Pushed

db48e03d5f22: Pushed

f49d1a66714f: Pushed

1a88bae59733: Pushed

5d4b948e9523: Pushed

5ab23d92cfc5: Pushed

v1: digest: sha256:7ef169aee875847d13a9960172c203150b666926d70286bdf3f254a87fb9802c size: 856

Result: Docker push successful.

Step 6 — Understand the push output

Output	Meaning
`Pushed`	Layer uploaded to the registry
`Mounted from ...`	Registry reused an already available layer
`v1`	Published image version
`digest: sha256:...`	Content-based identifier returned for the pushed artifact

The `Mounted from library/wordpress` message does not mean you pushed WordPress as your application. Docker Hub reused a matching layer from its existing registry storage.

Step 7 — Verify the Docker Hub image

Docker Hub image page:

python-challenge — Docker Hub Tags

The successful CLI push confirms that Docker Hub accepted the image. A separate screenshot confirming the v1 tag on the Docker Hub page has not yet been provided.

To verify the image through the Docker Hub interface, open the link and confirm the v1 tag is displayed.

Step 8 — Docker logout

Recommended cleanup command:

docker logout

Purpose: Remove locally stored Docker Hub login credentials for this registry.

This command was recommended after pushing, but its execution has not yet been confirmed.

5. Final task results

Docker Hub Push

SUCCESS

Image version

v1

Image ID

7ef169aee875

Repository created

Docker image tagged

Docker Hub authentication successful

Image push successful

6. Important commands to remember

Command	Purpose
`docker image ls`	List local Docker images
`docker tag`	Assign another name or tag to an image
`docker login`	Authenticate with Docker registry
`docker push`	Upload image to a registry
`docker pull`	Download image from a registry
`docker logout`	Remove local registry login credentials

7. Real-time interview questions and answers

Q1. What is Docker Hub?

Docker Hub is a container image registry used to store and distribute Docker images.

Q2. What is the purpose of `docker tag`?

The `docker tag` command creates another reference to an existing Docker image. For Docker Hub, the name generally includes the username, repository, and version tag.

Q3. What is the difference between `docker push` and `docker pull`?

`docker push` uploads an image to a registry. `docker pull` downloads an image from a registry.

Q4. Why do we need Docker login?

Authentication verifies that we have permission to push images into the target Docker Hub repository.

Q5. What does `Layer already exists` mean?

It means Docker Hub already stores a matching layer, so the layer does not need to be uploaded again.

Q6. What is a Docker image tag?

A tag is a readable identifier for an image version, such as `v1`, `v2`, or `build-25`.

Q7. Can the same Docker image be pushed to Docker Hub and AWS ECR?

Yes. We can tag the same local image using different registry names and push it to both registries, provided we have the required permissions.

Q8. How can you verify an image push?

I check the `docker push` output for successful completion and the digest, then verify that the image tag appears in the registry. I can also pull the published image as an additional test.

8. Interview project explanation

In my Docker challenge, I created a customized Python Flask Docker image using Python 3.12 Slim as the base image.

After testing the application successfully on EC2, I created a public repository in Docker Hub.

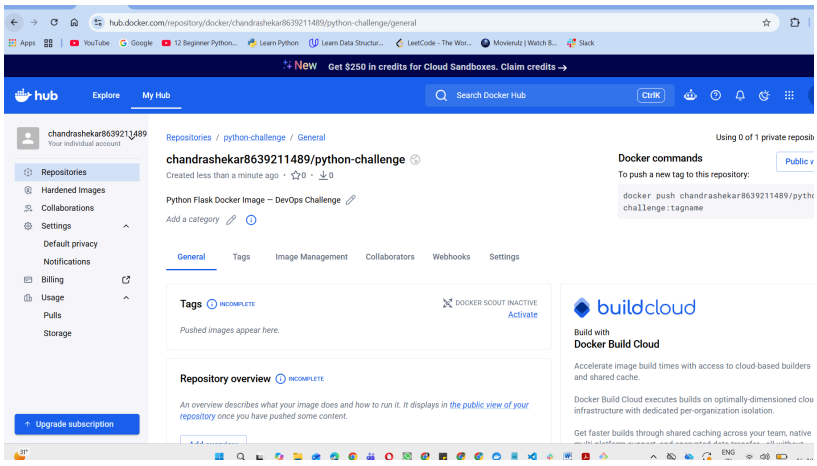
I tagged the local image with my Docker Hub username and repository name, authenticated using Docker CLI, and pushed the image.

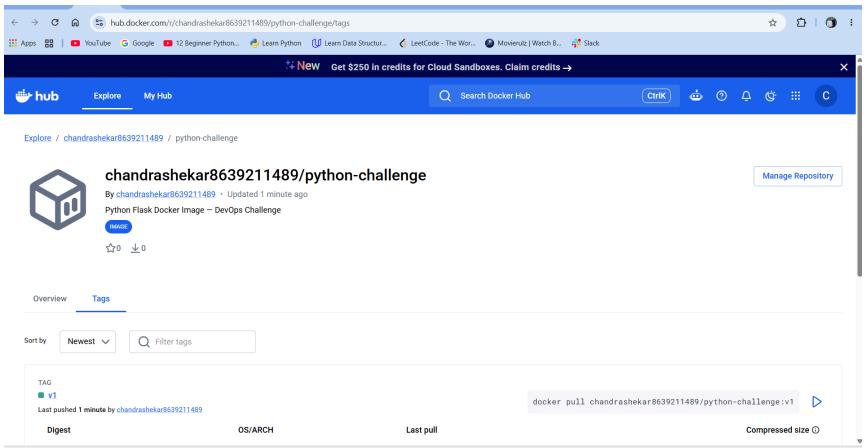
Docker Hub successfully accepted the image with the v1 tag and returned an image digest.

This exercise helped me understand Docker image tagging, registry authentication, image layers, and publishing images for reuse across environments.

9. Conclusion

Docker Challenge — Task 2 push completed successfully.





2. Push the same image to Amazon ECR.

objective

The objective is to push the same customized Python Flask Docker image created in Task 1 and published to Docker Hub in Task 2 into a private Amazon ECR repository.

We reused the existing image, `python-challenge:v1`, without rebuilding it.

2. Environment details

Component	Actual configuration
Cloud platform	AWS
Operating system	Ubuntu EC2
AWS account ID	`986881030460`
AWS region	`us-east-1`
EC2 IAM role	`Docker-ECR-Role`
AWS CLI	Installed and working
Docker version	29.9.0
Local image	`python-challenge:v1`

Image ID	`7ef169aee875`
ECR repository	`python-challenge`
Image tag	`v1`
Repository visibility	Private

3. Architecture and workflow

Ubuntu EC2 Server

python-challenge:v1

IAM Role Authentication

Docker-ECR-Role

Docker Login and Tag

AWS ECR repository URI

Amazon ECR Private Repository

python-challenge:v1

4. Step-by-step implementation

Step 1 — Verify AWS IAM credentials

First, we checked whether the EC2 instance had permission to access AWS.

aws sts get-caller-identity

Actual output:

{

```
  "UserId": "AROA6LRVRPE6IFQD4UW3D:i-0a28aceca9ceddc75",
  "Account": "986881030460",
  "Arn": "arn:aws:sts::986881030460:assumed-role/Docker-ECR-Role/i-0a28aceca9ceddc75"
}
```

Purpose: Verify the AWS account and IAM role being used by the EC2 instance.

Result: EC2 was successfully using `Docker-ECR-Role`, so no AWS access keys were required.

Step 2 — Verify the Docker image

Checked whether the image from Task 1 existed locally.

```
docker image ls python-challenge
```

Actual output:

```
IMAGE                ID             DISK USAGE   CONTENT SIZE
python-challenge:v1  7ef169aee875   198MB        48.3MB
```

Result: The existing Python image was available and ready to be pushed.

Step 3 — Check whether the ECR repository exists

Executed:

```
aws ecr describe-repositories \
  --repository-names python-challenge \
  --region us-east-1
```

Actual error:

```
RepositoryNotFoundException
The repository with name 'python-challenge' does not exist
```

Reason: We had not yet created an ECR repository for this Python application.

Solution: Create a private ECR repository named `python-challenge`.

Step 4 — Create an ECR repository

Initially, we executed:

```
aws ecr create-repository \
  --repository-name python-challenge \
  --region us-east-1 \
  --image-scanning-configuration scanOnPush=true
```

AWS returned:

```
AccessDeniedException
Not authorized to perform: ecr:CreateRepository
```

Reason: The existing `Docker-ECR-Role` did not have permission to create the `python-challenge` repository.

Step 5 — Resolve the IAM permission issue

Opened the AWS Console:

IAM → Roles → Docker-ECR-Role → Add permissions → Create inline policy

Created an IAM policy named:

`Allow-Python-Challenge-ECR-Create`

Policy JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCreatePythonChallengeRepository",
      "Effect": "Allow",
      "Action": "ecr:CreateRepository",
      "Resource": "arn:aws:ecr:us-east-1:986881030460:repository/python-challenge"
    }
  ]
}
```

Purpose: Grant the minimum required permission to create the specific ECR repository, instead of allowing unrestricted repository creation.

After attaching the policy, retried:

```
aws ecr create-repository \
  --repository-name python-challenge \
  --region us-east-1 \
  --image-scanning-configuration scanOnPush=true
```

Actual successful response included:


```
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:986881030460:repository/python-challenge",
    "registryId": "986881030460",
    "repositoryName": "python-challenge",
    "repositoryUri": "986881030460.dkr.ecr.us-east-1.amazonaws.com/python-challenge",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": true
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```


ECR repository created successfully.

Note: The repository-level `scanOnPush` setting was present in the creation response. Automatic vulnerability scanning was not tested in this task, and registry-level scanning rules can control actual scanning behavior.

Step 6 — Authenticate Docker to AWS ECR

Executed:

```
aws ecr get-login-password --region us-east-1 | \
```

```
docker login \
  --username AWS \
  --password-stdin \
  986881030460.dkr.ecr.us-east-1.amazonaws.com
```

Actual output:

```
WARNING! Your credentials are stored unencrypted in
'/root/.docker/config.json'.

Login Succeeded
```

Result: Docker successfully authenticated to the private ECR registry.

The AWS CLI uses the EC2 IAM role to obtain a temporary registry authentication password.

Step 7 — Tag the existing Docker image

Executed:

```
docker tag python-challenge:v1 \
  986881030460.dkr.ecr.us-east-1.amazonaws.com/python-challenge:v1
```

Verified:

```
docker image ls 986881030460.dkr.ecr.us-east-1.amazonaws.com/python-challenge
```

Actual output:

```
IMAGE                                                          ID             DISK USAGE   CONTENT SIZE
986881030460.dkr.ecr.us-east-1.amazonaws.com/python-challenge:v1   7ef169aee875
198MB         48.3MB
```

Purpose: Assign the full AWS ECR registry path to the existing Docker image.

Because the image ID remained `7ef169aee875`, this was the same image used in Tasks 1 and 2.

Step 8 — Push the Docker image to ECR

Executed:

```
docker push \
```

986881030460.dkr.ecr.us-east-1.amazonaws.com/python-challenge:v1

Actual output:

The push refers to repository
[986881030460.dkr.ecr.us-east-1.amazonaws.com/python-challenge]

75b6a36c64a1: Pushed
55de361ca85b: Pushed
44136fa355b3: Pushed
ecc510c1e359: Pushed
6912e23eb445: Pushed
db48e03d5f22: Pushed
f49d1a66714f: Pushed
bbc5a1223747: Pushed
5d4b948e9523: Pushed
1a88bae59733: Pushed
5ab23d92cfc5: Pushed

v1: digest:
sha256:7ef169aee875847d13a9960172c203150b666926d70286bdf3f254a87fb9802c size: 856

Result: All required image layers and the image manifest were successfully pushed to AWS ECR.

Step 9 — Verify the image in ECR

Executed:

```
aws ecr describe-images \
  --repository-name python-challenge \
  --image-ids imageTag=v1 \
  --region us-east-1 \
  --query 'imageDetails[].{Tags:imageTags,Status:imageStatus,Digest:imageDigest}'
```

Actual output:

```
[
```

```
  {
    "Tags": ["v1"],
    "Status": "ACTIVE",
    "Digest":
"sha256:7ef169aee875847d13a9960172c203150b666926d70286bdf3f254a87fb9802c"
  }
]
```


Result: AWS ECR confirmed that the image was stored successfully and its status was `ACTIVE`.

Step 10 — Log out from ECR

Executed:

docker logout 986881030460.dkr.ecr.us-east-1.amazonaws.com

Actual output:

Removing login credentials for
986881030460.dkr.ecr.us-east-1.amazonaws.com

Purpose: Remove the locally stored ECR registry login credentials.

5. Final task verification

AWS ECR — python-challenge:v1

ACTIVE

IAM role verified

Docker image verified

ECR repository created

IAM permission issue resolved

Docker login successful

Image tagged and pushed

AWS CLI image verification successful

Docker logout successful

6. Docker Hub vs Amazon ECR

You have now pushed the same customized Python Docker image into two registries.

Feature	Docker Hub	Amazon ECR
Provider	Docker	AWS
Repository	`python-challenge`	`python-challenge`
Image version	`v1`	`v1`
Access	Public repository	Private repository
Authentication	Docker Hub login	AWS IAM role and ECR login
Image digest	Same	Same
Push result	Successful	Successful

Same Python Docker Image

python-challenge:v1

Docker Hub

Public image registry

Amazon ECR

Private AWS registry

7. Errors encountered and solutions

Error	Reason	Solution
`RepositoryNotFoundException`	ECR repository did not exist	Created `python-challenge`
`AccessDeniedException`	IAM role lacked `ecr:CreateRepository`	Added a repository-specific inline IAM policy
Docker credential warning	Docker stored login credentials locally	Logged out after completing the push

8. Important commands to remember

Command	Purpose
`aws sts get-caller-identity`	Identify the AWS account and IAM identity
`aws ecr describe-repositories`	Check ECR repository details
`aws ecr create-repository`	Create an ECR repository
`aws ecr get-login-password`	Retrieve ECR authentication password

`docker login`	Authenticate with container registry
`docker tag`	Add an ECR-qualified image reference
`docker push`	Upload the Docker image to ECR
`aws ecr describe-images`	Verify pushed images
`docker logout`	Remove stored registry login credentials

9. Real-time interview questions and answers

Q1. What is Amazon ECR?

Amazon Elastic Container Registry is a managed AWS service used to store, manage, and distribute container images.

Q2. How do you push a Docker image to ECR?

First, I create an ECR repository, authenticate Docker using the AWS CLI, tag the image with the ECR repository URI, push the image, and verify it using AWS CLI.

Q3. Why do we attach an IAM role to EC2?

An IAM role provides temporary AWS credentials to applications and commands running on EC2 without storing long-lived AWS access keys on the server.

Q4. What is the difference between Docker Hub and Amazon ECR?

Docker Hub is a general-purpose container registry, while Amazon ECR is an AWS-managed container registry integrated with IAM and other AWS services.

Q5. Why did the ECR repository creation fail initially?

The IAM role lacked the `ecr:CreateRepository` permission for the new repository. I resolved it by adding a policy scoped to the required repository ARN.

Q6. Why is `docker tag` required?

The image needs a repository-qualified name so Docker knows which registry and repository should receive it.

Q7. Does `docker tag` rebuild an image?

No. It creates another reference to the same existing image.

Q8. Why do we use `--password-stdin` with Docker login?

It passes the registry authentication password through standard input rather than supplying the secret directly as a command-line argument.

Q9. How do you verify an image was successfully pushed?

I check the successful `docker push` output and then use `aws ecr describe-images` to confirm the tag, image status, and digest.

Q10. Can the same image be pushed to multiple registries?

Yes. I can assign different registry-specific tags to the same image and push it to both Docker Hub and Amazon ECR without rebuilding it.

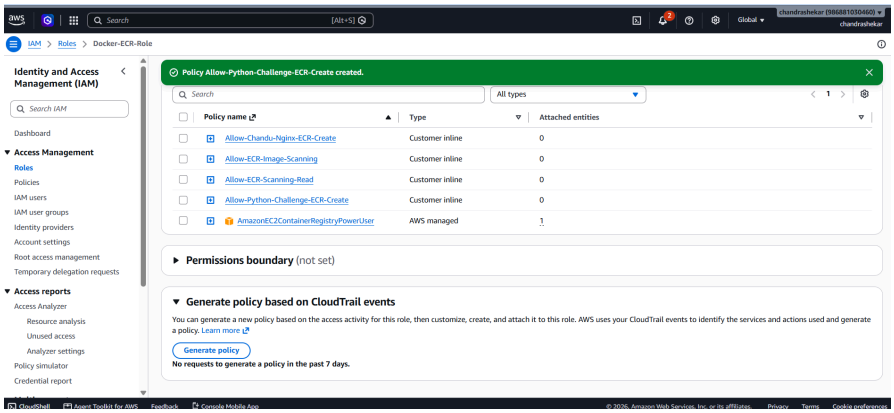
10. Interview project explanation

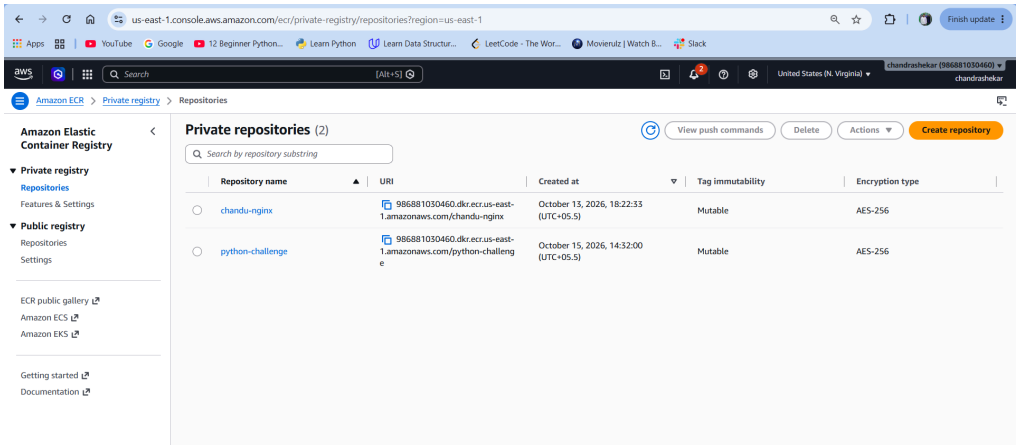
> In my Docker challenge, I worked with a Python Flask application and created a customized Docker image using Python 3.12 Slim.
>
> After successfully building and testing the Docker image, I published it to Docker Hub and Amazon ECR.
>
> For ECR, I used an EC2 IAM role to authenticate to AWS, created a private repository, and resolved an IAM permission issue related to repository creation.
>
> I then authenticated Docker with ECR, tagged the existing Python Docker image using the ECR repository URI, and pushed the image.
>
> Finally, I verified the image using AWS CLI. The ECR image status was ACTIVE, and its digest matched the Docker Hub image, confirming that I had successfully published the same image to both registries.

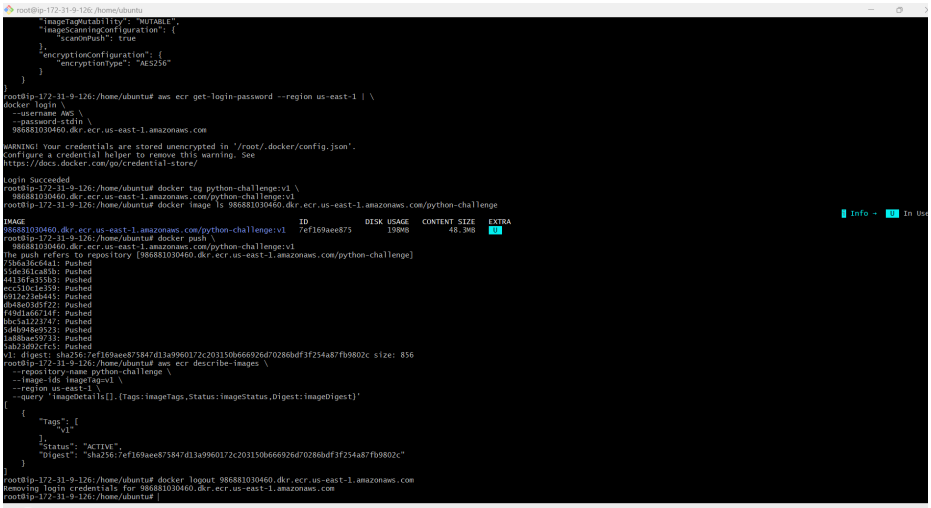
11. Conclusion

Docker Challenge — Task 3 completed successfully

You successfully created an Amazon ECR private repository, configured IAM permissions, authenticated Docker, pushed the Python application image, verified its presence, and logged out securely.







3. Provision one EC2 instance using Terraform and install Jenkins.

1. Objective

Provision a new Ubuntu EC2 instance using Terraform, configure its security group, install Java 21 and Jenkins, and access Jenkins through a web browser.

2. Architecture

Windows Laptop

|

| SSH (Port 22)

v

AWS EC2 — Existing Docker Server

|

| Terraform

v

AWS EC2 — New Jenkins Server

|

+-- Ubuntu 24.04

+-- Java 21

+-- Jenkins LTS

|

v

Jenkins Dashboard (Port 8080)

3. Infrastructure Details

Component	Value
AWS Region	`us-east-1`
EC2 Instance ID	`i-09f7a437cdd488896`

EC2 Instance Name	`Docker-Challenge-Terraform-Jenkins`
Instance Type	`c7i-flex.large`
Operating System	Ubuntu 24.04 LTS
AMI ID	`ami-0fa5967347d08d2df`
VPC ID	`vpc-09ace864d17ee9449`
Subnet ID	`subnet-00444f9336cffeedf`
Security Group ID	`sg-065508ee0c23fc3a9`
Public IP	`3.238.142.244`
Private IP	`172.31.2.37`
Storage	20 GiB gp3
Key Pair	`my_server`
Java Version	OpenJDK 21.0.12.1

Jenkins Port	8080

4. Install Terraform on Existing EC2 Server

Terraform was installed on the original Docker EC2 instance using the official HashiCorp APT repository.

Verify Terraform:

terraform version

Verified version:

Terraform v1.16.6

5. Create Terraform Project Directory

mkdir -p /home/ubuntu/docker-challenge/task4-terraform

cd /home/ubuntu/docker-challenge/task4-terraform

Our project contains:

task4-terraform/
├── main.tf
├── variables.tf
├── terraform.tfvars
├── outputs.tf
├── .terraform/
├── .terraform.lock.hcl
└── terraform.tfstate

6. Create `variables.tf`

This file declares the input variables.

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
```

```
}

variable "ami_id" {
  description = "Ubuntu AMI ID"
  type        = string
  default     = "ami-0fa5967347d08d2df"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "c7i-flex.large"
}

variable "subnet_id" {
  description = "Public subnet ID"
  type        = string
  default     = "subnet-00444f9336cffeedf"
}

variable "key_name" {
  description = "Existing EC2 key pair"
  type        = string
  default     = "my_server"
}

variable "allowed_ip" {
  description = "Trusted public IP CIDR"
  type        = string
}
```

Purpose: Variables allow us to reuse Terraform configuration without changing the resource definitions.

7. Create `terraform.tfvars`

This file provides values for Terraform variables.

```
aws_region    = "us-east-1"
ami_id        = "ami-0fa5967347d08d2df"
instance_type = "c7i-flex.large"
subnet_id     = "subnet-00444f9336cffeedf"
key_name      = "my_server"
allowed_ip    = "103.160.27.159/32"
```

We initially selected `t3.medium`, but AWS rejected it under the account's Free Tier restrictions. We changed the instance type to `c7i-flex.large`.

Note: Replace `allowed_ip` with your current trusted public IP if it changes.

8. Create `main.tf`

This is the main Terraform configuration.

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_security_group" "jenkins_sg" {
  name_prefix = "docker-challenge-jenkins-"
```

```
  description = "Security group for Terraform Jenkins EC2"
  vpc_id      = "vpc-09ace864d17ee9449"

  tags = {
    Name = "Docker-Challenge-Jenkins-SG"
  }
}

resource "aws_vpc_security_group_ingress_rule" "ssh" {
  security_group_id = aws_security_group.jenkins_sg.id
  description       = "SSH from trusted IP"
  cidr_ipv4         = var.allowed_ip
  from_port         = 22
  to_port           = 22
  ip_protocol       = "tcp"
}

resource "aws_vpc_security_group_ingress_rule" "jenkins" {
  security_group_id = aws_security_group.jenkins_sg.id
  description       = "Jenkins from trusted IP"
  cidr_ipv4         = var.allowed_ip
  from_port         = 8080
  to_port           = 8080
  ip_protocol       = "tcp"
}

resource "aws_vpc_security_group_egress_rule" "outbound" {
  security_group_id = aws_security_group.jenkins_sg.id
  description       = "Allow outbound IPv4"
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1"
}
```

```
resource "aws_instance" "jenkins" {
  ami                    = var.ami_id
  instance_type          = var.instance_type
  subnet_id              = var.subnet_id
  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]
  key_name               = var.key_name
  associate_public_ip_address = true

  root_block_device {
    volume_size = 20
    volume_type = "gp3"
  }

  tags = {
    Name    = "Docker-Challenge-Terraform-Jenkins"
    Project = "Docker-Challenge"
  }
}
```

9. Create `outputs.tf`

Outputs display important information after Terraform provisions the infrastructure.

```
output "jenkins_instance_id" {
  description = "Jenkins EC2 instance ID"
  value       = aws_instance.jenkins.id
}

output "jenkins_public_ip" {
  description = "Jenkins EC2 public IP address"
  value       = aws_instance.jenkins.public_ip
}

output "jenkins_url" {
  description = "Jenkins web interface URL"
```

```
  value       = "http://${aws_instance.jenkins.public_ip}:8080"
}

output "jenkins_ssh_command" {
  description = "SSH command for the Jenkins server"
  value       = "ssh -i my_server.pem ubuntu@${aws_instance.jenkins.public_ip}"
}
```

10. Initialize Terraform

terraform fmt

terraform init

terraform validate

Purpose of each command:

Command	Purpose
`terraform fmt`	Format Terraform code
`terraform init`	Download AWS provider and initialize working directory
`terraform validate`	Check configuration validity
`terraform plan`	Preview planned infrastructure changes
`terraform apply`	Create or modify infrastructure
`terraform output`	Display output values

`terraform state list`	List resources tracked in Terraform state

Successful initialization:

Terraform has been successfully initialized!

AWS provider installed:

hashicorp/aws v6.69.0

Successful validation:

Success! The configuration is valid.

11. Configure IAM Permissions

The Terraform execution environment used the EC2 IAM role:

Docker-ECR-Role

We used separate inline policies for Terraform resource discovery and provisioning.

These permissions included actions such as:

ec2:DescribeVpcs
ec2:DescribeSubnets
ec2:DescribeSecurityGroups
ec2:DescribeInstances
ec2:DescribeTags
ec2:DescribeInstanceAttribute
ec2:RunInstances
ec2:CreateSecurityGroup
ec2:AuthorizeSecurityGroupIngress
ec2:AuthorizeSecurityGroupEgress
ec2:CreateTags
ec2:TerminateInstances
ec2:DeleteSecurityGroup

Security note: These are representative permissions used during the lab, not a complete production least-privilege policy. Production IAM policies should scope resource-changing actions appropriately.

12. Run Terraform Plan

terraform plan

Original output:

Plan: 5 to add, 0 to change, 0 to destroy.

The five resources were:

1. EC2 instance
2. Security group
3. SSH ingress rule
4. Jenkins ingress rule
5. Outbound rule

13. Provision EC2

terraform apply

Type `yes` when prompted.

During the lab, AWS rejected `t3.medium` because it was not Free Tier eligible for the account.

We checked supported types using:

```
aws ec2 describe-instance-types \
  --region us-east-1 \
  --filters "Name=free-tier-eligible,Values=true" \
  --query 'InstanceTypes[].{Type:InstanceType,vCPU:VCpuInfo.DefaultVCpus,MemoryMiB:MemoryInfo.SizeInMiB}' \
  --output table
```

We updated `terraform.tfvars` to use:

instance_type = "c7i-flex.large"

Then generated a new plan and reapplied the configuration.

14. Troubleshooting During Provisioning

We encountered the following issues and resolved them.

Issue	Root cause	Resolution
Missing AWS provider	Terraform wasn't initialized	Ran `terraform init`
`iam:ListAttachedRolePolicies` denied	EC2 role lacked IAM inspection permission	Inspected policies through AWS Console
EC2 instance type rejected	`t3.medium` not eligible	Changed to `c7i-flex.large`
`ec2:DescribeTags` denied	Missing IAM read permission	Added permission
`ec2:DescribeInstanceAttribute` denied	Missing IAM read permission	Added permission
EC2 instance tainted	Terraform apply failed after resource creation	Verified instance and ran `terraform untaint`

Important Terraform learning: A failed apply does not necessarily mean AWS created nothing. Always inspect both AWS resources and Terraform state before retrying.

We verified the instance using:

```
aws ec2 describe-instances \
  --instance-ids i-09f7a437cdd488896 \
  --region us-east-1 \
  --query
'Reservations[].Instances[].{InstanceId:InstanceId,State:State.Name,PublicIP:PublicIpAddress,Type:InstanceType}' \
```

--output table

Terraform tracked all five resources:

terraform state list

We then removed the tainted status:

terraform untaint aws_instance.jenkins

A subsequent `terraform plan` showed only pending output changes, without infrastructure replacement.

15. Verify Terraform Outputs

terraform output

Actual result:

jenkins_instance_id = "i-09f7a437cdd488896"

jenkins_public_ip = "3.238.142.244"

jenkins_ssh_command = "ssh -i my_server.pem ubuntu@3.238.142.244"

jenkins_url = "http://3.238.142.244:8080"

16. Connect to EC2 Using SSH

From Windows Git Bash:

cd ~/Downloads

chmod 400 my_server.pem

ssh -i my_server.pem ubuntu@3.238.142.244

Successful connection:

Welcome to Ubuntu 24.04.5 LTS

ubuntu@ip-172-31-2-37:~$

17. Install Java 21

Jenkins requires a supported Java runtime.

sudo apt update

sudo apt install -y fontconfig openjdk-21-jre

Verify:

java -version

Actual output:

openjdk version "21.0.12.1" 2026-08-18

18. Install Jenkins LTS

Add the Jenkins signing key:

sudo mkdir -p /etc/apt/keyrings

sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

Configure the repository:

echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" \
  | sudo tee /etc/apt/sources.list.d/jenkins.list

Install Jenkins:

sudo apt update

sudo apt install -y jenkins

Verify the service:

sudo systemctl status jenkins --no-pager

Actual result:

jenkins.service - Jenkins Continuous Integration Server

Loaded: loaded (...; enabled; preset: enabled)
Active: active (running)

19. Unlock and Configure Jenkins

Open the Jenkins web interface:

`http://3.238.142.244:8080`

Retrieve the initial administrator password locally:

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Then complete the initial Jenkins setup:

1. Enter the administrator password.
2. Select Install suggested plugins.
3. Create an administrator account.
4. Finish the setup wizard.
5. Open the Jenkins dashboard.

Your browser screenshot confirmed the Jenkins dashboard was accessible.

20. Useful Jenkins Commands

Command	Purpose
`sudo systemctl status jenkins`	Check Jenkins status
`sudo systemctl start jenkins`	Start Jenkins
`sudo systemctl stop jenkins`	Stop Jenkins
`sudo systemctl restart jenkins`	Restart Jenkins
`sudo systemctl enable jenkins`	Enable startup on boot

`sudo journalctl -u jenkins -n 100 --no-pager`	View Jenkins service logs
`sudo ss -tulnp | grep 8080`	Check whether port 8080 is listening
`java -version`	Verify Java

21. Interview Questions and Answers

Q1. What is Terraform?

Terraform is an Infrastructure as Code tool used to create and manage infrastructure through configuration files.

Q2. Why did you use Terraform instead of creating EC2 manually?

Terraform makes infrastructure repeatable, version-controlled, and easier to manage consistently.

Q3. What is the difference between `terraform plan` and `terraform apply`?

`terraform plan` previews changes; `terraform apply` executes them.

Q4. What is Terraform state?

Terraform state records the resources Terraform manages and maps them to actual infrastructure.

Q5. What is a Terraform provider?

A provider is a plugin that enables Terraform to interact with services such as AWS.

Q6. What happens if `terraform apply` fails halfway?

Some resources may already exist. We must inspect the state and actual infrastructure before retrying.

Q7. What is Terraform taint?

A tainted resource is marked for replacement. `terraform untaint` removes that marker when retaining the existing resource is appropriate.

Q8. Why do we need Java for Jenkins?

Jenkins runs on Java, so a supported Java runtime must be available.

Q9. Why do we open port 8080?

Jenkins uses port 8080 by default for its web interface.

Q10. How do you troubleshoot Jenkins when it doesn't start?

Check the service status and logs:

sudo systemctl status jenkins

sudo journalctl -u jenkins -n 100 --no-pager

Then investigate Java compatibility, configuration, resources, and port conflicts.

22. Security and Cleanup

This lab restricted SSH and Jenkins access to a configured public IP. However, Jenkins was accessed through HTTP rather than HTTPS; production deployments should use TLS, and ideally avoid exposing Jenkins directly to the public internet.

After completing Task 5, you can remove the Terraform-managed resources from the original Terraform project directory using:

terraform plan -destroy

terraform destroy

Review the proposed destruction carefully before approving it. Do not delete your state files while infrastructure is still running.

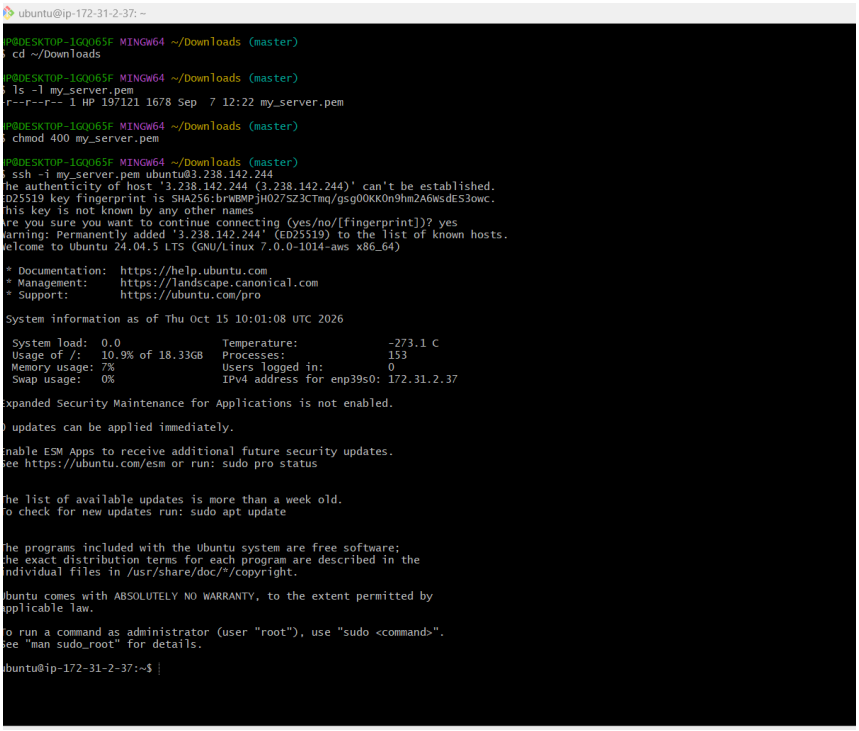
Also, avoid committing `terraform.tfstate`, Terraform plan files, or private SSH keys to a public GitHub repository.

23. Final Result

Task 4: Successfully completed.

We successfully provisioned a new Ubuntu EC2 instance through Terraform, configured its networking and security group, installed Java 21, installed Jenkins LTS, and accessed the Jenkins dashboard through a browser.

This task demonstrated real-world skills in Infrastructure as Code, AWS EC2, IAM permissions, Terraform troubleshooting, Linux administration, and Jenkins installation.



```
ubuntu@ip-172-31-2-37: ~
Need to get 53.9 MB of archives.
After this operation, 54.9 MB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 ne
t-tools amd64 2.10-0.1ubuntu4.4 [204 kB]
Get:2 https://pkg.jenkins.io/debian-stable binary/ jenkins 2.580.1 [53.7 MB]
Fetched 53.9 MB in 1s (65.9 MB/s)
Selecting previously unselected package net-tools.
(Reading database ... 94072 files and directories currently installed.)
Preparing to unpack .../net-tools_2.10-0.1ubuntu4.4_amd64.deb ...
Unpacking net-tools (2.10-0.1ubuntu4.4) ...
Selecting previously unselected package jenkins.
Preparing to unpack .../jenkins_2.580.1_all.deb ...
Unpacking jenkins (2.580.1) ...
Setting up net-tools (2.10-0.1ubuntu4.4) ...
Setting up jenkins (2.580.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service → /u
sr/lib/systemd/system/jenkins.service.
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

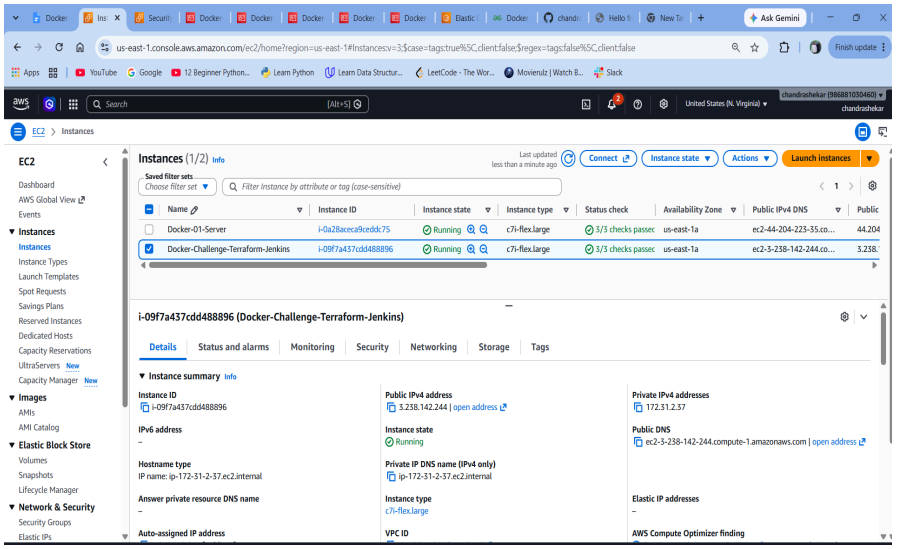
No services need to be restarted.

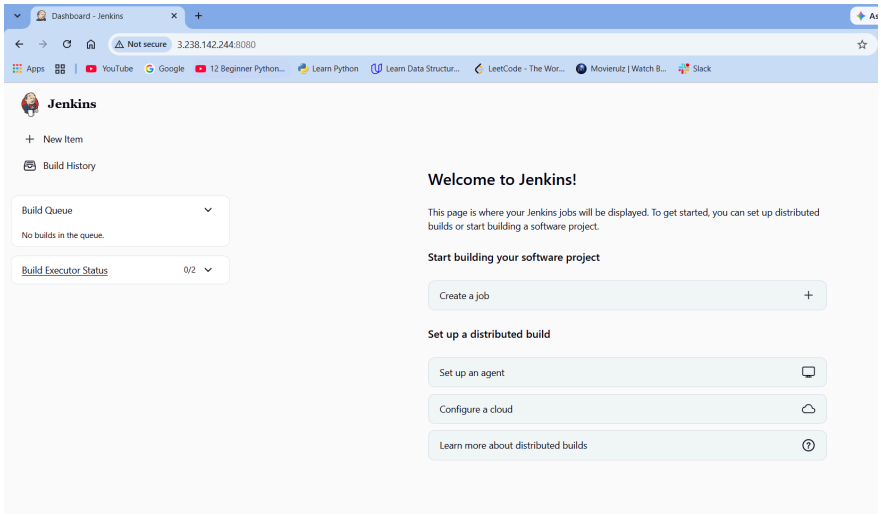
No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-2-37:~$ jenkins --version
2.580.1
ubuntu@ip-172-31-2-37:~$ sudo systemctl status jenkins --no-pager
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: e
nabled)
     Active: active (running) since Thu 2026-10-15 10:04:38 UTC; 12s ago
   Main PID: 4123 (java)
      Tasks: 49 (limit: 3712)
     Memory: 544.0M (peak: 549.5M)
        CPU: 12.978s
     CGroup: /system.slice/jenkins.service
             └─4123 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java…

Oct 15 10:04:35 ip-172-31-2-37 jenkins[4123]: [LF]> This may also be found a…ord
Oct 15 10:04:35 ip-172-31-2-37 jenkins[4123]: [LF]>
Oct 15 10:04:35 ip-172-31-2-37 jenkins[4123]: [LF]> ************************…***
Oct 15 10:04:35 ip-172-31-2-37 jenkins[4123]: [LF]> ************************…***
Oct 15 10:04:35 ip-172-31-2-37 jenkins[4123]: [LF]> ************************…***
Oct 15 10:04:38 ip-172-31-2-37 jenkins[4123]: 2026-10-15 10:04:38.634+0000 […ion
Oct 15 10:04:38 ip-172-31-2-37 jenkins[4123]: 2026-10-15 10:04:38.650+0000 […ing
Oct 15 10:04:38 ip-172-31-2-37 systemd[1]: Started jenkins.service - Jenkin…ver.
Oct 15 10:04:38 ip-172-31-2-37 jenkins[4123]: 2026-10-15 10:04:38.838+0000 […ler
Oct 15 10:04:38 ip-172-31-2-37 jenkins[4123]: 2026-10-15 10:04:38.838+0000 […  #1
Hint: Some lines were ellipsized, use -l to show in full.
ubuntu@ip-172-31-2-37:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
be06003300de40839ecb38641ae40ba1
ubuntu@ip-172-31-2-37:~$
```





4. **Create one Jenkins job to build and push the Docker image to Docker Hub.**
    - **Source: https://github.com/betawins/Python-app.git**

1. Objective

Create a Jenkins Declarative Pipeline that automatically downloads a Python Flask application from GitHub, builds a Docker image, and pushes the image to Docker Hub using securely stored Jenkins credentials.

Final result: Jenkins successfully published the Docker image with tag `build-4` to Docker Hub.

2. Project Architecture

GitHub Repository

betawins/Python-app

Jenkins on AWS EC2

Git checkout → Source verification

Docker Engine

Build and tag the Python image

Jenkins Credentials

Docker Hub authentication

Docker Hub

python-challenge:build-4

3. Environment Details

Component	Configuration
Cloud platform	AWS
Region	`us-east-1`
Operating system	Ubuntu 24.04
EC2 instance type	`c7i-flex.large`
Instance ID	`i-09f7a437cdd488896`
Jenkins server IP	`3.238.142.244`
Jenkins port	`8080`
Java	OpenJDK 21

Docker	Docker Engine
Git	2.43.0
Jenkins job	`Docker-Challenge-Python-CICD`
GitHub branch	`main`
Docker Hub username	`chandrashekar863921189`
Jenkins credential ID	`dockerhub-creds`

4. Step 1 — Connect to Jenkins EC2

Opened Git Bash on Windows and connected using the existing private key.

```
cd ~/Downloads

chmod 400 my_server.pem

ssh -i my_server.pem ubuntu@3.238.142.244
```

Successful connection:

```
Welcome to Ubuntu 24.04.5 LTS

ubuntu@ip-172-31-2-37:~$
```

Purpose: Access the EC2 instance provisioned using Terraform in Task 4.

5. Step 2 — Install Docker

Initially, Docker was not installed on the new Jenkins EC2 server.

Verified with:

```
docker --version
```

Initial result:

```
Command 'docker' not found
```

Installed Docker using:

```
sudo apt update

sudo apt install -y docker.io
```

Started and enabled the Docker service:

```
sudo systemctl enable --now docker
```

Verified:

```
docker --version

sudo systemctl status docker --no-pager
```

Actual service result:

```
Active: active (running)

Main PID: 5436 (dockerd)
```

Explanation: Docker Engine is responsible for building, tagging, and pushing container images. Jenkins will execute Docker commands during the pipeline.

6. Step 3 — Grant Docker Access to Jenkins

By default, Jenkins runs under the Linux user `jenkins`. That user needs access to the Docker daemon to build container images.

Added Jenkins to the Docker group:

```
sudo usermod -aG docker jenkins
```

Restarted Jenkins:

```
sudo systemctl restart jenkins
```

Verified group membership:

```
id jenkins
```

Actual result:

```
uid=111(jenkins) gid=113(jenkins) groups=113(jenkins),114(docker)
```

Tested Docker access as Jenkins:

```
sudo -u jenkins docker ps
```

Actual output:

```
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

This confirmed the Jenkins user could communicate with the Docker daemon.

Security consideration: Membership in the Docker group effectively provides root-level capabilities on the host. This setup was used for a dedicated training environment. Production Jenkins workloads should run on appropriately isolated agents.

7. Step 4 — Create Docker Hub Access Token

Opened Docker Hub:

Docker Hub Personal Access Tokens

Created a personal access token for Jenkins with repository read/write permissions.

Field	Value
Description	`jenkins-python-challenge`

Permissions	Read & Write
Purpose	Authenticate Docker image pushes
Delete permission	Not required

The token was copied securely and was not written directly into the Jenkins pipeline.

Why use a token? It provides a separate credential for automation that can be revoked without changing the Docker Hub account password.

8. Step 5 — Store Docker Hub Credentials in Jenkins

Opened Jenkins and navigated to:

Manage Jenkins → Credentials → System → Global credentials → Add Credentials

Selected Username with password.

Field	Value
Kind	Username with password
Scope	Global
Username	`chandrashekar8639211489`
Password	Docker Hub personal access token

ID	`dockerhub-creds`
Description	`Docker Hub credentials for Jenkins pipeline`

Saved the credential.

Verified that `dockerhub-creds` appeared in the Jenkins credentials list.

Important: Never place Docker Hub access tokens directly into source code, GitHub repositories, or Jenkins console output.

9. Step 6 — Create Jenkins Pipeline Job

Opened the Jenkins dashboard and selected New Item.

Entered the job name:

```
Docker-Challenge-Python-CICD
```

Selected Pipeline and clicked OK.

Under the Pipeline section, selected Pipeline script and added an initial test pipeline:

```
pipeline {
    agent any

    stages {
        stage('Git Checkout') {
            steps {
                echo 'Downloading source code'
            }
        }
```

```
    }
}
```

Clicked Save → Build Now.

Build #1 completed successfully:

```
Downloading source code
Finished: SUCCESS
```

Learning point: The `echo` step prints a message but does not actually clone a GitHub repository. This first build only verified Jenkins Pipeline execution.

10. Step 7 — Configure GitHub Checkout

Updated the pipeline to clone the application repository and verify source files.

```
pipeline {
    agent any

    stages {
        stage('Git Checkout') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/betawins/Python-app.git'
            }
        }

        stage('Verify Source Code') {
            steps {
                sh '''
```

```
                echo "Checking downloaded files..."
                pwd
                ls -la
            '''
        }
    }
  }
}
```

Executed Build #2.

The console confirmed:

```
Checking out Revision a9c18a887f134751f06bab9eb128a4387b42c141
```

Jenkins workspace:

```
/var/lib/jenkins/workspace/Docker-Challenge-Python-CICD
```

Files verified:

```
.git/
.gitignore
Dockerfile
Jenkinsfile
app.py
requirements.txt
templates/
```

Final result:

```
Finished: SUCCESS
```

Learning point: Jenkins downloads source code into its workspace before executing build commands.

11. Step 8 — Build Docker Image with Jenkins

Added two additional stages: `Docker Build` and `Verify Docker Image`.

The image build command was:

```
docker build -t
chandrashekar8639211489/python-challenge:build-${BUILD_NUMBER} .
```

Explanation:

Part	Purpose
`docker build`	Build a container image
`-t`	Assign image name and tag
`python-challenge`	Docker Hub repository
`BUILD_NUMBER`	Jenkins-provided build number
`.`	Current workspace as build context

Executed Build #3.

The console output confirmed:

```
Successfully built ec59a2bdd4e0
```

```
Successfully tagged chandrashekar8639211489/python-challenge:build-3

Docker image build completed.
```

Verified the locally built image:

```
docker images chandrashekar8639211489/python-challenge
```

The image was available locally under tag `build-3`.

Note: The source repository's Dockerfile uses `python:3.6`, an unsupported Python release. The lab build succeeded, but a production pipeline should use a supported runtime and test application compatibility.

12. Step 9 — Complete Jenkins Declarative Pipeline

We added Docker Hub authentication and image pushing using the stored Jenkins credential.

This is the final pipeline script used for the task.

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = 'chandrashekar8639211489/python-challenge'
        IMAGE_TAG  = "build-${BUILD_NUMBER}"
    }

    stages {
        stage('Git Checkout') {
            steps {
                git branch: 'main',
```

```
            url: 'https://github.com/betawins/Python-app.git'
        }
    }

    stage('Verify Source Code') {
        steps {
            sh '''
                echo "Checking application files..."
                ls -la
            '''
        }
    }

    stage('Docker Build') {
        steps {
            sh '''
                docker build -t "$IMAGE_NAME:$IMAGE_TAG" .
            '''
        }
    }

    stage('Verify Docker Image') {
        steps {
            sh '''
                docker images "$IMAGE_NAME"
```

```
            '''
        }
    }

    stage('Push to Docker Hub') {
        steps {
            withCredentials([
                usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_TOKEN'
                )
            ]) {
                sh '''
                    set +x
                    export DOCKER_CONFIG="
```
$(mktemp -d)"trap'dockerlogout > /dev/null2 > &1||true;rm -rf"
```
DOCKER_CONFIG''' EXIT

                    echo "$DOCKER_TOKEN" | docker login \
                        --username "$DOCKER_USER" \
                        --password-stdin

                    docker push "$IMAGE_NAME:IMAGE_TAG"
                '''
            }
```

```
            }
        }
    }

    post {
        success {
            echo 'SUCCESS: Python image pushed to Docker Hub!'
        }

        failure {
            echo 'FAILED: Check Jenkins console output.'
        }
    }
}
```

Explanation of the Jenkins pipeline

Jenkins concept	Explanation
`pipeline`	Defines the complete workflow
`agent any`	Executes on an available Jenkins agent
`environment`	Defines variables accessible to the pipeline

`stages`	Groups major CI/CD activities
`stage`	Represents one activity, such as checkout or build
`steps`	Commands executed within a stage
`git`	Downloads code from GitHub
`sh`	Runs Linux shell commands
`withCredentials`	Makes stored credentials temporarily available
`docker login`	Authenticates with Docker Hub
`docker push`	Publishes the image
`post`	Executes actions after pipeline completion

The temporary `DOCKER_CONFIG` directory is used to isolate Docker authentication data for the build and remove it afterward.

13. Step 10 — Run Complete Jenkins Pipeline

Saved the pipeline and clicked Build Now.

The next Jenkins execution was Build #4, so the image tag became:

chandrashekar8639211489/python-challenge:build-4

The pipeline executed the intended workflow:

Git Checkout

↓

Verify Source Code

↓

Docker Build

↓

Verify Docker Image

↓

Push to Docker Hub

The published image was independently verified through Docker Hub.

14. Step 11 — Verify Docker Image in Docker Hub

Opened:

Docker Hub — Python Challenge Tags

The repository displayed the newly pushed image.

Docker Hub Publishing Result

Verified

Image tag

build-4

OS / Architecture

linux/amd64

Compressed size

333.24 MB

Publishing status

Successful

Optional verification command:

```
docker pull chandrashekar8639211489/python-challenge:build-4
```

This command downloads the published image from Docker Hub.

15. Troubleshooting and Challenges

Problem or warning	Reason	Solution
Docker not found	Fresh EC2 instance	Installed `docker.io`
Jenkins cannot access Docker	Linux user permissions	Added Jenkins to Docker group
GitHub files not downloading	Initial pipeline only used `echo`	Added actual `git` checkout
Docker Hub authentication needed	Jenkins must push to a remote registry	Created Docker Hub token and Jenkins credential
Docker legacy builder warning	Build used the older Docker builder	Consider Docker Buildx/BuildKit

Python 3.6 in Dockerfile	Repository uses an old runtime	Upgrade to a supported Python version after compatibility testing

No failed Jenkins build was shown during the checkout and Docker build steps. The earlier `echo` test was intentionally limited to testing pipeline execution.

16. Real-Time Interview Questions and Answers

Q1. Explain the CI/CD pipeline you implemented.

I implemented a Jenkins Declarative Pipeline to download a Python Flask application from GitHub, build a Docker image, authenticate with Docker Hub using Jenkins credentials, and push the image with a unique Jenkins build-number tag.

Q2. Why did you install Docker on the Jenkins server?

Jenkins executes Docker commands such as `docker build` and `docker push`. Docker Engine must be accessible to the executor running these commands.

Q3. Why did you add Jenkins to the Docker group?

Jenkins runs under a separate Linux account. Docker group membership allowed it to communicate with the Docker daemon in this lab.

Q4. Why did you create Docker Hub credentials in Jenkins?

To avoid hardcoding authentication secrets in the pipeline script or GitHub repository.

Q5. What is the purpose of `withCredentials`?

It temporarily provides selected Jenkins credentials to pipeline steps through environment variables, with masking for recognized secret values in console output.

Q6. What is `BUILD_NUMBER`?

It is a Jenkins environment variable identifying the current build number. We used it to create tags such as `build-3` and `build-4`.

Q7. What is the difference between `docker build` and `docker push`?

`docker build` creates an image locally. `docker push` uploads a tagged image to a registry such as Docker Hub.

Q8. What is a Jenkins workspace?

It is the directory where Jenkins checks out source code and executes build steps.

Our workspace was:

/var/lib/jenkins/workspace/Docker-Challenge-Python-CICD

Q9. Why do we use multiple pipeline stages?

Stages organize the workflow, make failures easier to identify, and improve the readability of execution logs.

Q10. What happens if Docker Hub login fails?

The image push should not proceed. We would verify the Jenkins credential ID, token permissions, account username, repository access, and token validity.

Q11. How do you avoid exposing Docker Hub passwords in Jenkins?

Use Jenkins Credentials, credentials binding, a scoped access token, and avoid printing secrets in the console output.

Q12. How would you improve this pipeline for production?

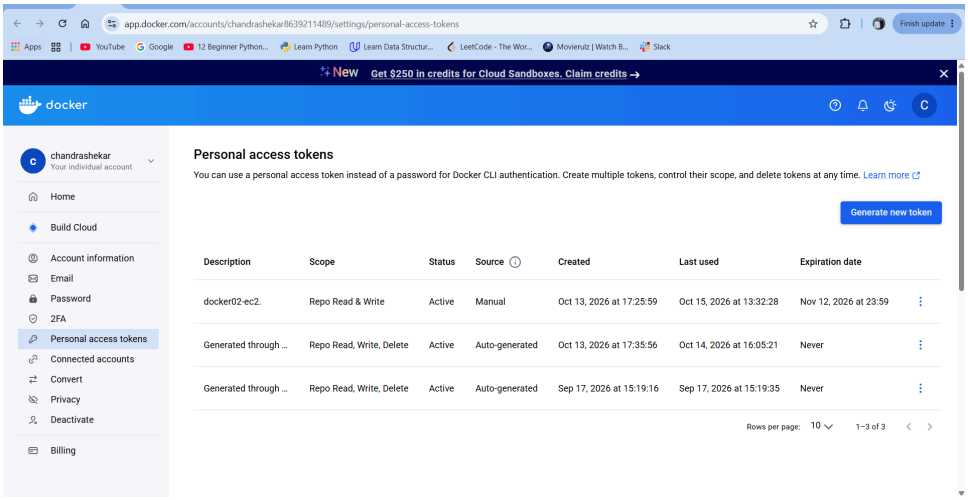
I would use supported Python base images, add application tests and security scanning, run builds on isolated Jenkins agents, use HTTPS, and configure automatic GitHub webhook triggers.

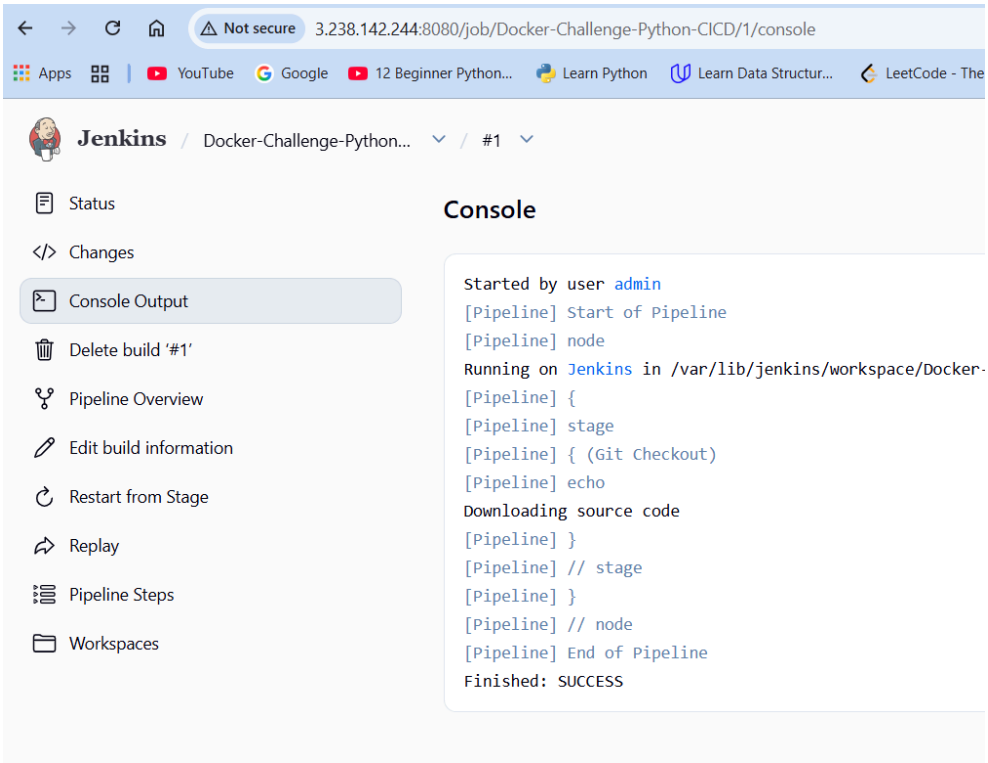
17. Important Security Notes

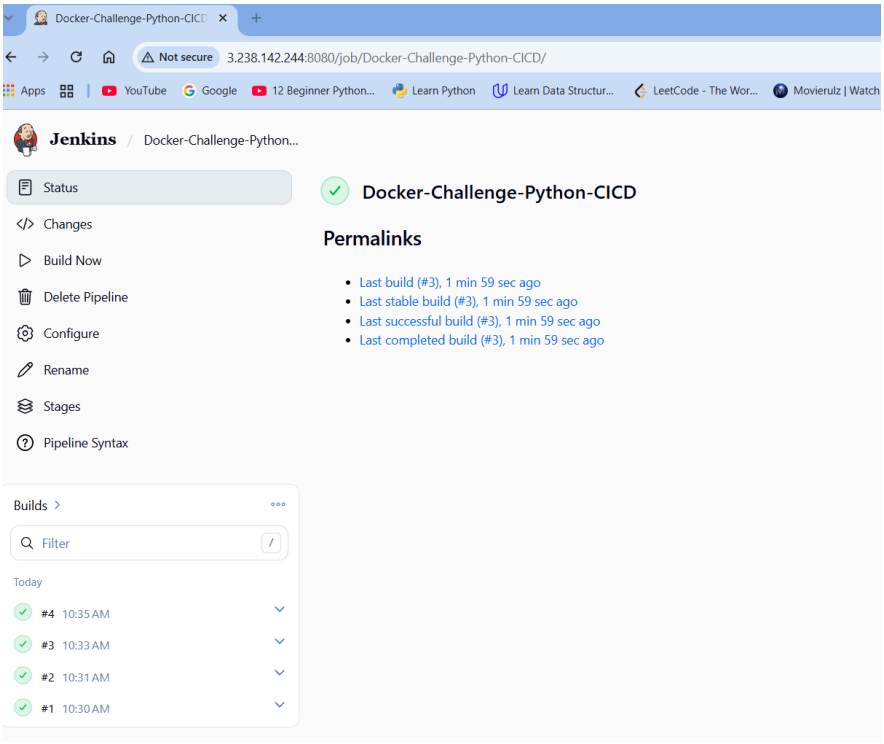
The Jenkins server was accessible at `http://3.238.142.244:8080`. Because HTTP does not encrypt web traffic, Jenkins should use HTTPS or an SSH tunnel, especially when administering credentials.

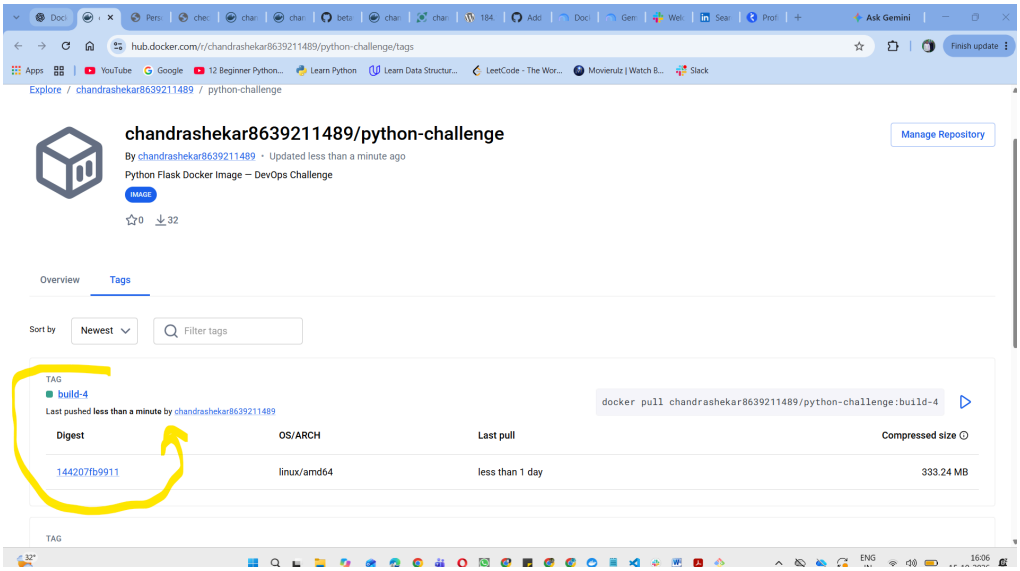
For a production environment, also restrict administrator permissions, keep plugins updated, isolate Docker builds, and use narrowly scoped registry tokens.

Your Terraform-managed EC2 server and associated AWS resources may incur charges while running. Preserve the Terraform project and state so you can safely destroy the lab infrastructure when it is no longer required.






Docker-Challenge-Python-CICD
Not secure
3.238.142.244:8080/job/Docker-Challenge-Python-CICD/
Apps
YouTube
Google
12 Beginner Python...
Learn Python
Learn Data Structur...
LeetCode - The Wor...
Movierulz | Watch
Jenkins
Docker-Challenge-Python...
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
Stages
Pipeline Syntax
Docker-Challenge-Python-CICD
Permalinks
Last build (#3), 1 min 59 sec ago
Last stable build (#3), 1 min 59 sec ago
Last successful build (#3), 1 min 59 sec ago
Last completed build (#3), 1 min 59 sec ago
Builds
Filter
Today
#4 10:35 AM
#3 10:33 AM
#2 10:31 AM
#1 10:30 AM


hub.docker.com/r/chandrashekar8639211489/python-challenge/tags
Explore / chandrashekar8639211489 / python-challenge
chandrashekar8639211489/python-challenge
By chandrashekar8639211489 • Updated less than a minute ago
Python Flask Docker Image — DevOps Challenge
IMAGE
Manage Repository
Overview
Tags
Sort by
Newest
Filter tags
TAG
build-4
Last pushed less than a minute by chandrashekar8639211489
docker pull chandrashekar8639211489/python-challenge:build-4
Digest
OS/ARCH
Last pull
Compressed size
144207fb9911
linux/amd64
less than 1 day
333.24 MB
TAG

📂 **Source Codes: https://github.com/betawins/docker-tasks.git**

- **From the frontend source code, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.**

Task: Create a custom Docker image for a static frontend application, run it using Nginx, test the website, and push the image to Docker Hub.

You've already completed the Docker build and verified the website in your browser. Only Docker Hub publishing and final verification remain.

Step	Activity	Status
1	Clone GitHub repository	Completed
2	Identify frontend source files	Completed
3	Create custom Dockerfile	Completed
4	Build Docker image	Completed
5	Run frontend Docker container	Completed
6	Test website in browser	Completed
7	Create Docker Hub repository and tag image	Pending

8	Push image to Docker Hub	Pending
9	Verify published image	Pending

Steps 1–6 — Commands already executed

```
# Clone the repository
git clone https://github.com/betawins/docker-tasks.git

# Open the frontend source directory
cd docker-tasks/Frontend_based_source

# Verify application files
ls -lah

# Build the custom Docker image
docker build -t frontend-challenge:v1 .

# Run the container
docker run -d \
  --name frontend-challenge-container \
  -p 8091:80 \
  frontend-challenge:v1

# Test the website
curl -I http://localhost:8091
```

Your test returned `HTTP/1.1 200 OK`, and the Amazon-style frontend loaded successfully at:

Frontend application — Port 8091

Step 7 — Create and tag Docker Hub image

First, create a public repository named `frontend-challenge` on Docker Hub.

Then run:

```
docker tag frontend-challenge:v1 \
  chandrashekar8639211489/frontend-challenge:v1
```

Verify:

```
docker images | grep frontend-challenge
```

Step 8 — Login and push

Log in using your Docker Hub username and access token. Docker may prompt you to use browser-based authentication.

```
docker login -u chandrashekar8639211489
```

Then push:

```
docker push chandrashekar8639211489/frontend-challenge:v1
```

Expected final output is similar to:

```
v1: digest: sha256:...
```

Step 9 — Verify the image

Open:

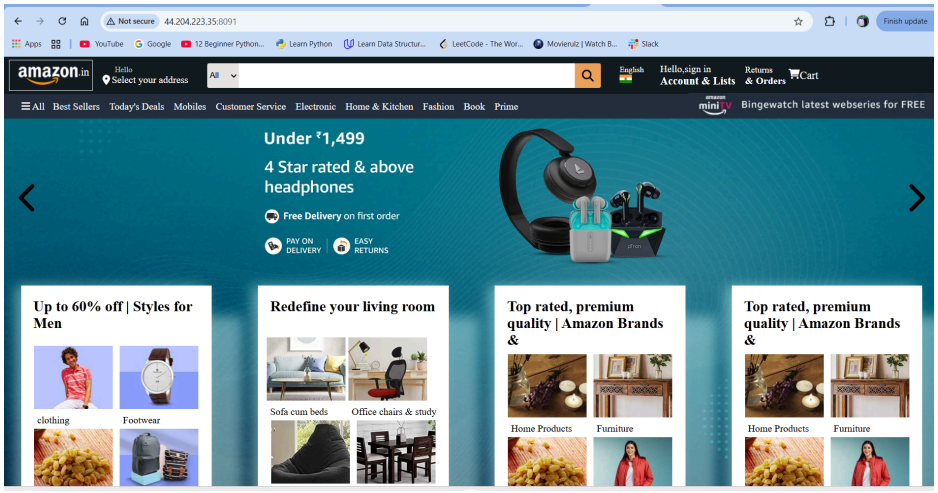
Docker Hub — Frontend Challenge Tags

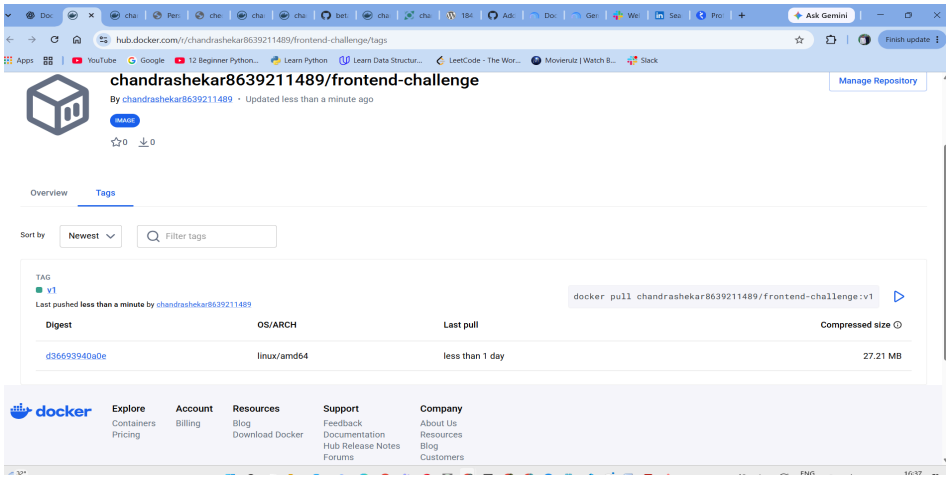
Check that tag `v1` appears.

You can also verify by pulling the image:

```
docker pull chandrashekar8639211489/frontend-challenge:v1
```

Your next action: Complete Steps 7 and 8, then send me the `docker push` output or a Docker Hub screenshot showing `v1`.





- **From the Java-based source code, write a Dockerfile, build, run, and push the image to the Docker registry.**

1. Project objective

The objective of Task 7 was to containerize a Java application using Docker and make the image available on Docker Hub.

We successfully built a Spring Boot application, ran it inside a container, verified it through a browser, and published the Docker image.

2. Project architecture

GitHub — Java source code

pom.xml + src/

Docker Stage 1 — Maven

Compile source code → Generate JAR

Docker Stage 2 — Java Runtime

Run Spring Boot JAR

AWS EC2 — Docker Container

Port 8092 → 8080

Docker Hub

java-challenge:v1

3. Environment details

Component	Value
Cloud	AWS EC2
Operating system	Ubuntu Linux
Public IP	`44.204.223.35`
Repository	betawins/docker-tasks
Application directory	`Java_based_source`

Framework	Spring Boot 2.2.4.RELEASE
Build tool	Maven
Java version	11
Application JAR	`spring-boot-web.jar`
Docker image	`java-challenge:v1`
Docker container	`java-challenge-container`
Host port	`8092`
Container port	`8080`

4. Step 1 — Clone the GitHub repository

The repository was already cloned in Task 6, so we reused it.

Original cloning command:

git clone https://github.com/betawins/docker-tasks.git

Open the Java application directory:

cd ~/docker-challenge/task6/docker-tasks/Java_based_source

Verify files:

ls -lah

Observed project structure:

```
Java_based_source/
├── pom.xml
└── src/
```

5. Step 2 — Identify the application type

Check Java project files:

```
find . -maxdepth 3 -type f | sort | head -60
```

Search for Maven, Gradle, JAR, and WAR files:

```
find . -maxdepth 3 \( \
  -name "pom.xml" \
  -o -name "build.gradle" \
  -o -name "build.gradle.kts" \
  -o -name "*.jar" \
  -o -name "*.war" \
\)
```

Actual result:

```
./pom.xml
```

Conclusion: This is a Maven-based Java project.

6. Step 3 — Identify Spring Boot

We searched the source code:

```
grep -R -n -E \
'@SpringBootApplication|public static void main|@RestController|@RequestMapping' \
src | head -40
```

Actual result:

```
src/main/java/com/hussain/StartApplication.java:9:@SpringBootApplication
src/main/java/com/hussain/StartApplication.java:20:    public static void main(String[] args)
```

This confirms that the project uses Spring Boot and has a Java main method.

Explanation

Java component	Purpose
`@SpringBootApplication`	Marks the Spring Boot application class
`main()`	Entry point of the Java application
`pom.xml`	Defines project dependencies and build configuration
Maven	Compiles and packages the Java application

7. Step 4 — Check Java version and dependencies

Run:

```
grep -n -E \
'java.version|maven.compiler|spring-boot|<packaging>|<artifactId>|<version>' \
pom.xml
```

Important results:

```
<artifactId>spring-boot-demo</artifactId>
<version>1.0</version>
<version>2.2.4.RELEASE</version>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
<java.version>11</java.version>
<finalName>spring-boot-web</finalName>
```

We identified:

- Java version: 11
- Spring Boot version: 2.2.4.RELEASE
- Maven artifact: spring-boot-demo
- Output JAR: spring-boot-web.jar

We also checked:

cat src/main/resources/application.properties

Actual result:

nothing here yet

Since no custom server port was configured, the application uses Spring Boot's default HTTP port, `8080`.

8. Step 5 — Create a multi-stage Dockerfile

We created a Dockerfile with two stages.

The first stage uses Maven to build the application. The second stage runs the compiled JAR with Java 11.

Stage 1: Build Java application

FROM maven:3.9-eclipse-temurin-11 AS build

WORKDIR /app

COPY pom.xml .

RUN mvn dependency:go-offline -B

COPY src ./src

RUN mvn clean package -DskipTests -B

Stage 2: Run Spring Boot application

```
FROM eclipse-temurin:11-jre

WORKDIR /app

COPY --from=build /app/target/spring-boot-web.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Dockerfile explanation

Instruction	Purpose
`FROM maven:3.9-eclipse-temurin-11`	Provides Maven and Java 11 for compiling
`AS build`	Names the build stage
`WORKDIR /app`	Creates/sets the working directory
`COPY pom.xml .`	Copies Maven configuration
`RUN mvn dependency:go-offline -B`	Pre-fetches Maven dependencies where possible
`COPY src ./src`	Copies application source code

`RUN mvn clean package -DskipTests -B`	Packages the application without running tests
`FROM eclipse-temurin:11-jre`	Uses a Java runtime for the final container
`COPY --from=build`	Copies the JAR from the build stage
`EXPOSE 8080`	Documents the application port
`ENTRYPOINT`	Starts the Java application

Why multi-stage Docker builds?

They separate compiling the application from running it. The final image does not need to include Maven and the complete source tree.

For production use, Java 11 and Spring Boot 2.2 should be reviewed and upgraded because these versions are outdated.

9. Step 6 — Build the Docker image

Run:

docker build -t java-challenge:v1 .

Verify:

docker images java-challenge

Actual result:

IMAGE ID DISK USAGE CONTENT SIZE

java-challenge:v1 471c723da2f6 475MB 130MB

Result: Java Docker image successfully created.

10. Step 7 — Run the Java container

Run:

```
docker run -d \
  --name java-challenge-container \
  -p 8092:8080 \
  java-challenge:v1
```

Command explanation

Option	Meaning
`docker run`	Creates and starts a container
`-d`	Detached/background mode
`--name`	Container name
`-p 8092:8080`	Host port 8092 maps to container port 8080
`java-challenge:v1`	Image to execute

Check container status:

```
docker ps -a --filter name=java-challenge-container
```

Check logs:

```
docker logs --tail 40 java-challenge-container
```

These commands help determine whether the Spring Boot application started correctly.

11. Step 8 — Verify the application locally

We tested the application using:

curl -I http://localhost:8092

Actual output:

HTTP/1.1 200
Content-Type: text/html;charset=UTF-8
Content-Language: en-US
Transfer-Encoding: chunked

Result: The application returned HTTP `200`, confirming it responded successfully.

12. Step 9 — Access the application in the browser

Opened AWS EC2 Security Group inbound rules and allowed:

Field	Value
Type	Custom TCP
Port	`8092`
Source	My IP
Description	Java application testing

Then opened:

Java Spring Boot Application

The browser displayed the Techie Horizon — DevOps Training Institute homepage.

This confirmed that the application was accessible externally through Docker port mapping.

13. Step 10 — Create Docker Hub repository

Opened:

Docker Hub Repositories

Created the repository using:

Field	Value
Repository	`java-challenge`
Visibility	Public
Description	`Spring Boot Java 11 application using multi-stage Docker build`

14. Step 11 — Tag the Docker image

docker tag java-challenge:v1 \
chandrashekar8639211489/java-challenge:v1

Verify:

docker images | grep java-challenge

Expected image names:

java-challenge:v1
chandrashekar8639211489/java-challenge:v1

Explanation: `docker tag` creates another reference to the same image using the Docker Hub repository name.

15. Step 12 — Push the image to Docker Hub

Authenticate:

docker login -u chandrashekar8639211489

Use the supported browser login or an access token when prompted. Never place the token directly in commands that will be saved in shell history.

Push:

docker push chandrashekar8639211489/java-challenge:v1

Your Docker Hub screenshot confirmed that the repository contains the `v1` tag.

16. Step 13 — Verify Docker Hub

Opened:

Java Challenge Docker Hub Tags

Verified that:

Field	Result
Repository	`chandrashekar8639211489/java-challenge`
Tag	`v1`
Push status	Successful
Image visibility	Available on Docker Hub

Optional verification:

docker pull chandrashekar8639211489/java-challenge:v1

This downloads the image from Docker Hub.

17. Important Docker commands

Command	Purpose
`docker build -t java-challenge:v1 .`	Build the Java image
`docker images`	List Docker images
`docker run -d -p 8092:8080 ...`	Start Java container
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker logs container-name`	Check container logs
`docker stop container-name`	Stop a container
`docker start container-name`	Start an existing container
`docker tag`	Tag image for registry

`docker login`	Authenticate with Docker Hub
`docker push`	Upload image
`docker pull`	Download image

18. DevOps interview questions and answers

Q1. What is a multi-stage Docker build?

A multi-stage Docker build uses multiple `FROM` instructions. One stage can compile the application, while another contains only the files required to run it.

Q2. Why use Maven in the Dockerfile?

Maven downloads dependencies, compiles Java code, and packages the Spring Boot application into a JAR.

Q3. What is the difference between JDK and JRE?

The JDK provides tools for developing and compiling Java applications. A JRE provides the runtime needed to execute Java applications.

Q4. What is the purpose of `pom.xml`?

`pom.xml` contains the Maven project's dependencies, build plugins, version details, and configuration.

Q5. What is the purpose of `ENTRYPOINT`?

It defines the executable that starts when the Docker container runs. Here, it executes `java -jar app.jar`.

Q6. Explain `-p 8092:8080`.

Port `8092` is on the EC2 host. Port `8080` is inside the container. Docker forwards traffic from the host port to the application port.

Q7. How do you troubleshoot a Java container that stops immediately?

Run:

docker ps -a

docker logs java-challenge-container

Check whether the Java process failed, whether the JAR exists, and whether the application configuration is correct.

Q8. What happens if the host port is already in use?

Docker cannot bind another container to the same host IP and port. We must stop the conflicting service or choose another available host port.

Q9. What is the difference between `docker tag` and `docker push`?

`docker tag` assigns another image reference locally. `docker push` uploads the referenced image to a registry.

Q10. How can you reduce a Java Docker image's size?

Use multi-stage builds, keep build tools out of the runtime stage, select an appropriate runtime image, and exclude unnecessary files.

Q11. Why do you test with `curl` before opening the browser?

It separates local application functionality from networking and security-group issues. If local HTTP works but the browser cannot connect, we can investigate firewall rules, public IPs, and port mappings.

Q12. How would you improve this project for production?

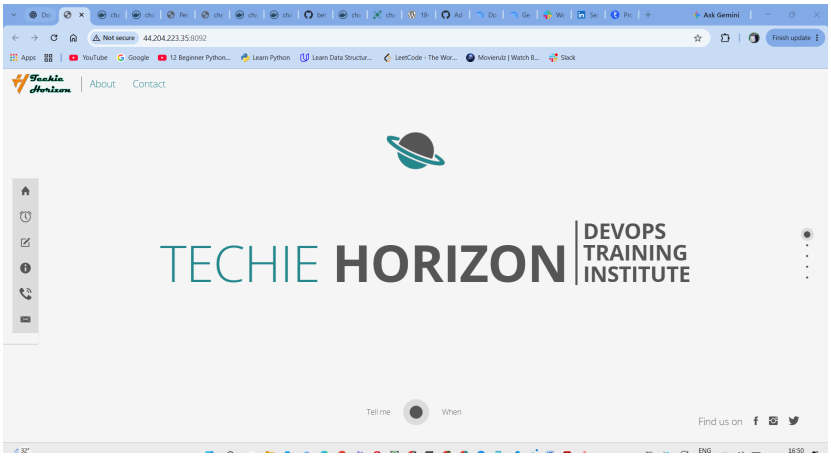
Upgrade unsupported framework/runtime versions, enable automated tests, add container image scanning, avoid running as root, pin base images, and use a CI/CD pipeline.

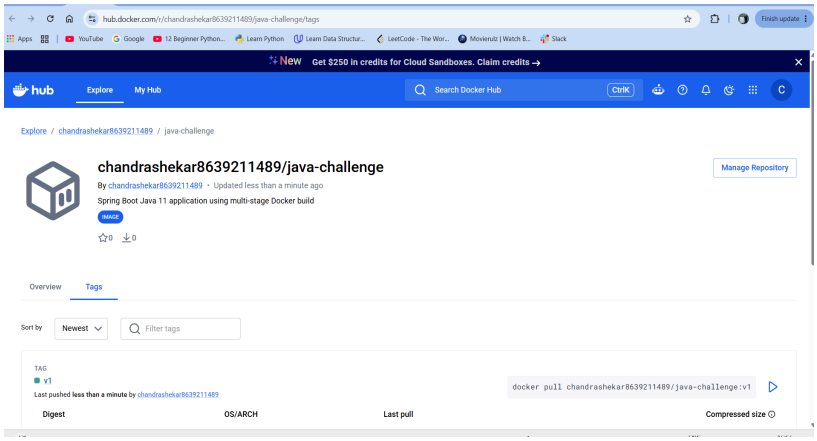
```
root@ip-172-31-9-126: ~/docker-challenge/task6/docker-tasks/Java_based_source
java-challenge:v1   471c723da2f6        475MB         130MB
root@ip-172-31-9-126:~/docker-challenge/task6/docker-tasks/Java_based_source# do
cker run -d \
  --name java-challenge-container \
  -p 8092:8080 \
  java-challenge:v1
530f5b43b49f193dd06d8f18583dc6ce4185fd972bc84a0c3e9411422be7cbd1
root@ip-172-31-9-126:~/docker-challenge/task6/docker-tasks/Java_based_source# do
cker ps -a --filter name=java-challenge-container
CONTAINER ID   IMAGE              COMMAND              CREATED          STATUS
          PORTS                                       NAMES
530f5b43b49f   java-challenge:v1   "java -jar app.jar"   13 seconds ago   Up 12
seconds   0.0.0.0:8092->8080/tcp, [::]:8092->8080/tcp   java-challenge-container
root@ip-172-31-9-126:~/docker-challenge/task6/docker-tasks/Java_based_source# do
cker logs --tail 40 java-challenge-container

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.2.4.RELEASE)

2026-10-15 11:18:29.958  INFO 1 --- [           main] com.hussain.StartApplicati
on            : Starting StartApplication v1.0 on 530f5b43b49f with PID 1 (/app
/app.jar started by root in /app)
2026-10-15 11:18:29.963  INFO 1 --- [           main] com.hussain.StartApplicati
on            : No active profile set, falling back to default profiles: defaul
t
2026-10-15 11:18:31.216  INFO 1 --- [           main] o.s.b.w.embedded.tomcat.To
mcatWebServer  : Tomcat initialized with port(s): 8080 (http)
2026-10-15 11:18:31.228  INFO 1 --- [           main] o.apache.catalina.core.Sta
ndardService   : Starting service [Tomcat]
2026-10-15 11:18:31.228  INFO 1 --- [           main] org.apache.catalina.core.S
tandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.30]
2026-10-15 11:18:31.284  INFO 1 --- [           main] o.a.c.c.C.[Tomcat].[localh
ost].[/]       : Initializing Spring embedded WebApplicationContext
2026-10-15 11:18:31.284  INFO 1 --- [           main] o.s.web.context.ContextLoa
der            : Root WebApplicationContext: initialization completed in 1222 ms
2026-10-15 11:18:31.446  INFO 1 --- [           main] o.s.s.concurrent.ThreadPoo
lTaskExecutor  : Initializing ExecutorService 'applicationTaskExecutor'
2026-10-15 11:18:31.523  INFO 1 --- [           main] o.s.b.a.w.s.WelcomePageHan
dlerMapping    : Adding welcome page template: index
2026-10-15 11:18:31.683  INFO 1 --- [           main] o.s.b.w.embedded.tomcat.To
mcatWebServer  : Tomcat started on port(s): 8080 (http) with context path ''
2026-10-15 11:18:31.687  INFO 1 --- [           main] com.hussain.StartApplicati
on            : Started StartApplication in 2.511 seconds (JVM running for 2.98
9)
root@ip-172-31-9-126:~/docker-challenge/task6/docker-tasks/Java_based_source# cu
rl -I http://localhost:8092
HTTP/1.1 200
Content-Type: text/html;charset=UTF-8
Content-Language: en-US
Transfer-Encoding: chunked
Date: Thu, 15 Oct 2026 11:19:09 GMT

root@ip-172-31-9-126:~/docker-challenge/task6/docker-tasks/Java_based_source#
```





- **From the Node.js-based source code, write a Dockerfile, build with tag `v1`, run, and push it to the Docker registry.**

.1. Objective

Create a Docker image for a React.js Zomato Clone application using a multi-stage Dockerfile, run it on AWS EC2, and push it to Docker Hub.

2. Project Details

Component	Value
GitHub	`betawins/docker-tasks`
Application	React.js — Zomato Clone
Node.js	20
Web server	Nginx
Image	`nodejs-challenge:v1`
Container	`nodejs-challenge-container`
Host port	`8093`

Docker Hub	`chandrashekar8639211489/nodejs-challenge:v1`

3. Step 1 — Open Application Directory

cd ~/docker-challenge/task6/docker-tasks/NodeJs_based_source

ls -lah

cat package.json

The application contains `package.json`, `package-lock.json`, `public/`, and `src/`.

4. Step 2 — Create Dockerfile

```
# Stage 1: Build React application
FROM node:20-alpine AS build

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY public ./public
COPY src ./src

RUN npm run build

# Stage 2: Serve application
FROM nginx:stable-alpine

COPY --from=build /app/build /usr/share/nginx/html
```

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Purpose: Node.js builds the React application, and Nginx serves the compiled website.

5. Step 3 — Build Docker Image

docker build -t nodejs-challenge:v1 .

Result: Image built successfully in 47 seconds.

6. Step 4 — Run Docker Container

```
docker run -d \
  --name nodejs-challenge-container \
  -p 8093:80 \
  nodejs-challenge:v1
```

Verify:

docker ps
curl -I http://localhost:8093

Result: `HTTP/1.1 200 OK`

7. Step 5 — Browser Verification

Allowed inbound TCP port `8093` from My IP in the EC2 security group.

Opened:

http://44.204.223.35:8093

Result: Zomato Clone homepage displayed successfully.

8. Step 6 — Push Image to Docker Hub

Created Docker Hub repository `nodejs-challenge`.

Tag image:

```
docker tag nodejs-challenge:v1 \
  chandrashekar8639211489/nodejs-challenge:v1
```

Login:

```
docker login -u chandrashekar8639211489
```

Push:

```
docker push chandrashekar8639211489/nodejs-challenge:v1
```

Verify:

[Docker Hub — Node.js Challenge]

Result: Tag `v1` published successfully.

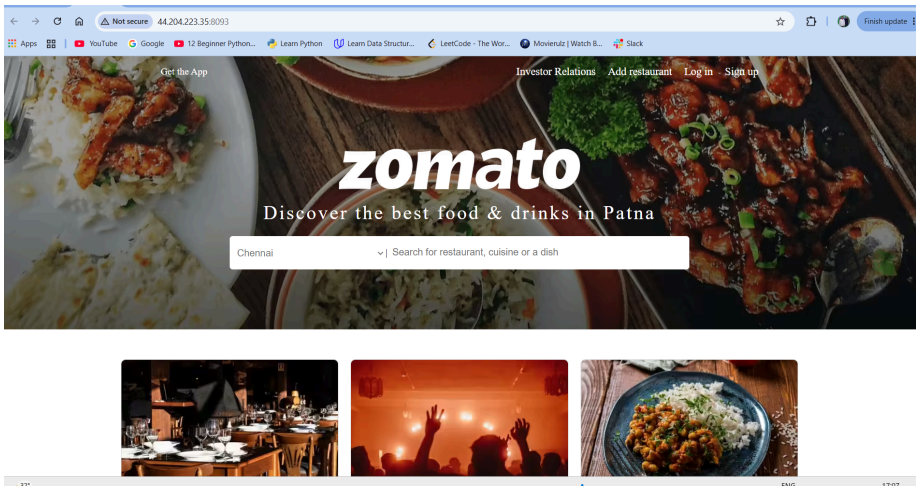
9. Important Interview Points

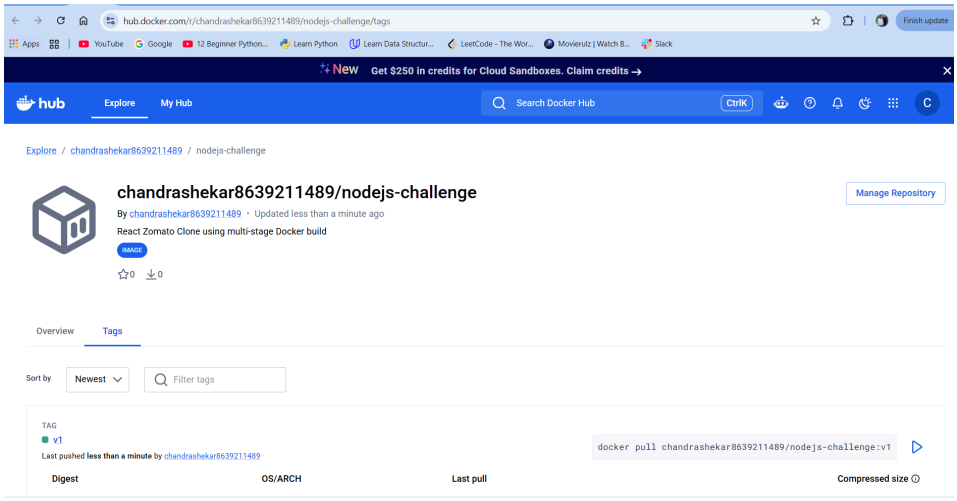
Question	Answer
Why multi-stage Dockerfile?	Keeps build tools out of the final image
Why Node.js?	Installs dependencies and builds React
Why Nginx?	Serves compiled static files
What is `npm ci`?	Installs dependencies using the lockfile

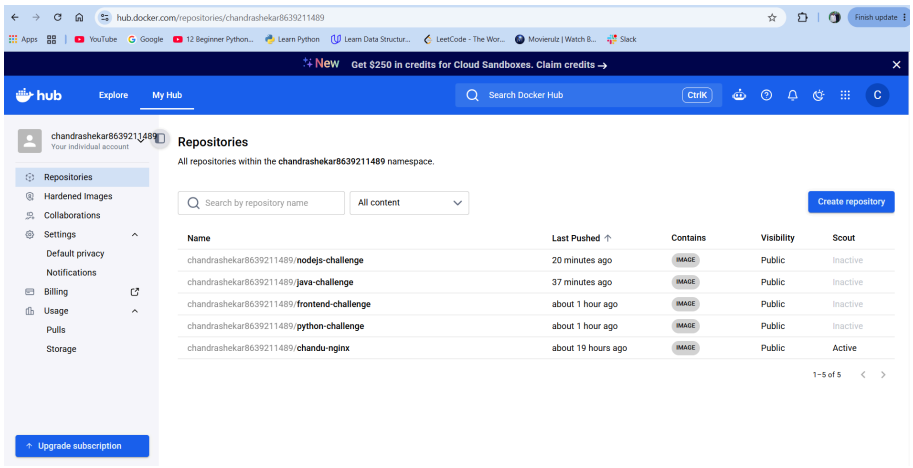
What is `npm run build`?	Generates production-ready React files
What is `8093:80`?	EC2 host port 8093 → container port 80
Why `docker push`?	Uploads the image to Docker Hub

10. Final Result

Task 8 successfully completed: Built, deployed, tested, and pushed the React.js Zomato Clone Docker image.







9.Write a docker-compose file to set up WordPress with a MySQL database.

1. Objective

Deploy WordPress and MySQL as separate Docker containers using Docker Compose, configure networking, and provide persistent data storage using Docker volumes.

2. Project Details

Component	Value

Server	AWS EC2 — Ubuntu
Docker Engine	29.9.0
Docker Compose	v5.6.0
WordPress image	`wordpress:php8.3-apache`
Database image	`mysql:8.0`
Host port	`8094`
WordPress container port	`80`
Database port	`3306` (internal only)
Project directory	`/home/ubuntu/docker-challenge/task9-wordpress`

3. Step 1 — Verify Docker Compose

docker --version

docker compose version

4. Step 2 — Create Project Directory

mkdir -p /home/ubuntu/docker-challenge/task9-wordpress

cd /home/ubuntu/docker-challenge/task9-wordpress

5. Step 3 — Create Environment File

Generated database passwords securely:

cat > .env <<EOF

MYSQL_ROOT_PASSWORD=$(openssl rand -hex 24) $MYSQL_PASSWORD = $(openssl rand -hex 24)

EOF

chmod 600 .env

Purpose: Store database credentials separately from the Compose YAML file. Do not upload `.env` to GitHub.

6. Step 4 — Create Docker Compose File

File: `compose.yaml`

```
services:
  db:
    image: mysql:8.0
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - wordpress_network
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -u root
-p\"$$MYSQL_ROOT_PASSWORD\" --silent"]
      interval: 10s
```

```
      timeout: 5s
      retries: 12
      start_period: 30s

  wordpress:
    image: wordpress:php8.3-apache
    restart: unless-stopped
    ports:
      - "8094:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: ${MYSQL_PASSWORD}
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wordpress_data:/var/www/html
    depends_on:
      db:
        condition: service_healthy
    networks:
      - wordpress_network

volumes:
  mysql_data:
  wordpress_data:

networks:
  wordpress_network:
    driver: bridge
```

Purpose: Run WordPress and MySQL together, connect them through a Docker network, and store their data in persistent volumes.

7. Step 5 — Validate and Start Containers

docker compose config --quiet

docker compose up -d

docker compose ps

Result: Both containers started successfully.

8. Step 6 — Verify WordPress

curl -I http://localhost:8094

Actual output:

HTTP/1.1 302 Found

Location: http://localhost:8094/wp-admin/install.php

The redirect indicated that WordPress was ready for its initial installation.

9. Step 7 — Complete WordPress Setup

Allowed TCP port `8094` from My IP in the AWS security group.

Opened:

WordPress Website

Completed the installation wizard, created the administrator account, and logged in successfully.

The WordPress dashboard was verified in the browser.

10. Step 8 — Verify MySQL

docker compose exec -T db sh -c \

'mysqladmin ping -h 127.0.0.1 -u root -p"$MYSQL_ROOT_PASSWORD"'

Actual result:

mysqld is alive

MySQL was confirmed healthy. The command may display a warning about supplying a password through a CLI argument.

Verified containers:

docker compose ps

Actual result:

task9-wordpress-db-1 Up (healthy)

task9-wordpress-wordpress-1 Up

11. Step 9 — Verify Docker Volumes

docker volume ls

The Compose configuration defines these persistent volumes:

- `task9-wordpress_mysql_data` — Stores MySQL database files.
- `task9-wordpress_wordpress_data` — Stores WordPress files.

Volume persistence across container recreation was configured but not independently tested during the task.

12. Important Docker Compose Commands

Command	Purpose
`docker compose up -d`	Start containers
`docker compose ps`	Check container status
`docker compose logs`	View logs
`docker compose restart`	Restart services

`docker compose down`	Remove containers and Compose network
`docker volume ls`	List Docker volumes
`docker compose down -v`	Delete containers and volumes — causes data loss

13. Interview Questions

Question	Short answer
What is Docker Compose?	A tool for defining and running multi-container applications
Why use two containers?	WordPress runs the website; MySQL manages database data
Why use Docker volumes?	To preserve data when containers are recreated
What is `depends_on`?	Controls service startup dependencies
What is a health check?	A test that checks service readiness or health
Why use Docker networks?	To enable communication between containers

Why is MySQL port 3306 not published?	WordPress can access MySQL internally without exposing the database to the internet
What does `8094:80` mean?	EC2 host port 8094 maps to WordPress container port 80
What does `.env` do?	Supplies configuration values and passwords to Compose

14. Final Result

Verification	Status
Docker Compose configuration	Passed
WordPress container	Running
MySQL container	Healthy
Database connectivity	Verified
WordPress installation	Successful
Administrator dashboard	Accessible
Persistent volumes	Configured

Task 9 successfully completed.

Interview-ready explanation

I deployed WordPress and MySQL using Docker Compose on AWS EC2. I created a Compose YAML file defining two services, a Docker network, persistent volumes, database environment variables, and a MySQL health check. After launching the containers, I verified database health, configured WordPress, and successfully accessed the administrator dashboard through the browser.

